

Creating an agent tool from an MCP server

Lab guide

watsonx Orchestrate Level 4



Version 2.2

July 2026

LEGAL NOTICES

This information was developed for products and services offered in the USA. IBM may not offer the products, services, or features discussed in this document in other countries. Consult your local IBM representative for information on the products and services currently available in your area. Any reference to an IBM product, program, or service is not intended to state or imply that only that IBM product, program, or service may be used. Any functionally equivalent product, program, or service that does not infringe any IBM intellectual property right may be used instead. However, it is the user's responsibility to evaluate and verify the operation of any non-IBM product, program, or service. IBM may have patents or pending patent applications covering subject matter described in this document. The furnishing of this document does not grant you any license to these patents. You can send license inquiries, in writing, to:

IBM Director of Licensing IBM Corporation
North Castle Drive, MD-NC119 Armonk, NY 10504-1785
United States of America

The following paragraph does not apply to the United Kingdom or any other country where such provisions are inconsistent with local law: INTERNATIONAL BUSINESS MACHINES CORPORATION PROVIDES THIS PUBLICATION “AS IS” WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. Some states do not allow disclaimer of express or implied warranties in certain transactions; therefore, this statement may not apply to you. This information could include technical inaccuracies or typographical errors. Changes are periodically made to the information herein; these changes will be incorporated in new editions of the publication. IBM may make improvements and/or changes in the product(s) and/or the program(s) described in this publication at any time without notice. Any references in this information to non-IBM websites are provided for convenience only and do not in any manner serve as an endorsement of those websites. The materials at those websites are not part of the materials for this IBM product and use of those websites is at your own risk. IBM may use or distribute any of the information you supply in any way it believes appropriate without incurring any obligation to you.

Information concerning non-IBM products was obtained from the suppliers of those products, their published announcements or other publicly available sources. IBM has not tested those products and cannot confirm the accuracy of performance, compatibility or any other claims related to non-IBM products. Questions on the capabilities of non-IBM products should be addressed to the suppliers of those products.

This information contains examples of data and reports used in daily business operations. To illustrate them as completely as possible, the examples include the names of individuals, companies, brands, and products. All names and references for organizations and other business institutions used in this deliverable's scenarios are fictional. Any match with real organizations or institutions is coincidental. All names and associated information for people in this deliverable's scenarios are fictional. Any match with a real person is coincidental.

TRADEMARKS

IBM, the IBM logo, and [ibm.com](https://www.ibm.com) are trademarks or registered trademarks of International Business Machines Corp., registered in many jurisdictions worldwide. Other product and service names might be trademarks of IBM or other companies. A current list of IBM trademarks is available on the web at “Copyright and trademark information” at <https://www.ibm.com/legal/copyright-trademark>.

Adobe, the Adobe logo, PostScript, and the PostScript logo are either registered trademarks or trademarks of Adobe Systems Incorporated in the United States, and/or other countries.

Java and all Java-based trademarks and logos are trademarks or registered trademarks of Oracle and/or its affiliates.

INFOBLOX is a trademark of Infoblox Inc. or its affiliated companies, registered in the United States and other countries.

Microsoft®, Azure, Windows, Windows NT, Windows Server, Microsoft® Excel®, Microsoft® Word®, Microsoft® Outlook®, Microsoft® Excel® logo, Microsoft® Word® logo, Microsoft® Outlook® logo, Microsoft® Windows App, Microsoft® Windows App logo and the Windows logo are trademarks of Microsoft® Corporation in the United States, other countries, or both.

Salesforce and Salesforce logo are registered trademarks of SALESFORCE, INC. (CORPORATION; CALIFORNIA, USA).

Slack is a registered trademark of Salesforce, Inc (CORPORATION; DELAWARE, USA).

Red Hat®, JBoss®, OpenShift®, Fedora®, Hibernate®, Ansible®, CloudForms®, RHCA®, RHCE®, RHCSA®, Ceph®, and Gluster® are trademarks or registered trademarks of Red Hat, Inc. or its subsidiaries in the United States and other countries.

Google, Google logo, Google Tasks, Google Cloud Platform, GMAIL, GMAIL logo are registered trademarks of GOOGLE LLC and this material is not endorsed by or affiliated with Google in any way.

Elasticsearch, Elastic search logo, Elastic, Elastic logo are registered trademarks of Elasticsearch BV (LIMITED LIABILITY COMPANY; NETHERLANDS) in United States, other countries or both.

Milvus, Milvus logo are registered trademarks of LF Projects, LLC (LIMITED LIABILITY COMPANY; DELAWARE, USA) in United States, other countries or both.

Meta, Meta logos, Llama, Llama logos are registered trademarks of META PLATFORMS, INC. (CORPORATION; California, USA) in United States, other countries or both.

Mistral, Mistral logo, Mistral AI, Mistral AI logo are registered trademarks of Mistral AI (SOCIÉTÉ PAR ACTIONS SIMPLIFIÉE (SAS); FRANCE) in United States, other countries or both.

PyCharm, PyCharm logo are registered trademarks of JetBrains s.r.o. (CORPORATION; CZECH REPUBLIC) in United States, other countries or both.

Bruno is a trademark held by Anoop M D. Bruno logo is sourced from OpenMoji. License: CC BY-SA 4.0

Apple, Apple logo, Apple store, App store, App store logo are registered trademarks of Apple Inc. (CORPORATION; CALIFORNIA, USA) in United States, other countries or both.

VMWARE, VMWARE logos are registered trademarks of VMware LLC (LIMITED LIABILITY COMPANY; Delaware, USA) in United States, other countries or both.

RANCHER, RANCHER logo, Rancher Desktop, Rancher Desktop logo are registered trademarks of SUSE LLC (LIMITED LIABILITY COMPANY; MASSACHUSETTS, USA) in United States, other countries or both.

Langfuse, Langfuse logo are registered trademarks of Langfuse GmbH (GESELLSCHAFT MIT BESCHRÄNKTER HAFTUNG (GMBH); Germany, Germany) in United States, other countries or both.

OpenMeteo is a registered trademark owned by Patrick Zippenfenig (Individual, Germany) in United States, other countries or both.

© Copyright International Business Machines Corporation 2025. This document may not be reproduced in whole or in part without the prior written permission of IBM.

Table of Contents

Introduction	1
About this lab	3
Lab overview.....	3
What you will learn	3
How to get support.....	3
Prerequisites	3
PART 1: Import MCP tools using watsonx Orchestrate	4
Create a Tavily connection	6
Using Local MCP servers to run tools.....	12
Create an agent	13
Configure an MCP Server.....	15
Select the tools to import from the MCP server	18
Adding more tools from your local MCP servers.....	21
Preview the tool within your agent.....	24
PART 2: Import MCP tools using watsonx Orchestrate Developer Edition (ADK)	26
Open your ADK environment	26
Create a Tavily connection	27
Create the connection	27
Configure the environments parameters.....	28
Specify the credentials to connect your tavily server.....	30
Import Tavily tools in a Tavily toolkit	32
Create the Tavily agent	32
Preview the agent	33
Next step	37
Summary	38
Appendix: Troubleshooting.....	39
Unable to create a connection	39
Appendix: Reset your environment	40

Introduction

MCP is a standard that allows agents to securely and flexibly interact with external tools and data sources through MCP servers. MCP enables agents to discover, connect to, and start external tools that are provided by MCP servers. These servers function as intermediaries, presenting various capabilities (such as searching the web, querying a database, sending email, or fetching web content) in a standardized manner. This process does not require creating custom APIs or manually setting up connections, making it a simpler and more efficient method for agents to use external tools.

With MCP, agents can discover and use these external tools during run time, enhancing their capabilities beyond their inherent features. All interactions with external tools are managed through a single protocol, which simplifies connections and improves efficiency. Moreover, MCP facilitates collaboration and tool sharing among agents from different vendors or platforms.

During this lab you will learn how to add a Tavily web search tool to an agent, imported using an MCP server provided by Tavily.

Tavily is a search engine and API specifically designed for AI agents and Retrieval-Augmented Generation (RAG) workflows. It provides real-time, accurate, and factual search results tailored for use with large language models (LLMs). Unlike traditional search engines, Tavily focuses on providing concise, ready-to-use information optimized for LLM context, streamlining the process of integrating dynamic web information into AI-driven solutions.

Warnings

When running MCP servers without proper sandboxing, they can execute arbitrary code on your system with the same permissions as the host process. This creates significant security risks.

Security Risks:

- System Access: Full access to files, network, and system resources.
- Code Execution: Can run any command on your machine.
- Prompt Injection: Malicious prompts could trigger unintended server actions.
- Data Exposure: Sensitive data may be accessed or leaked.

Best Practices:

- Ensure that the MCP server is legitimate and safe for your agents to use.
- Run servers in VMs or isolated environments.
- Review code before installation.
- Limit permissions to minimum required.
- Monitor server activity.

Note: Watsonx Orchestrate runs local MCP servers in an isolated VM.

About this lab

Lab overview

This lab will teach you how you can import external tools from Model Context Protocol (MCP) servers and add them to your agents to enhance the agent's ability to accomplish tasks.

This lab is composed of two independent parts:

- Part 1: Creating MCP tools using low-code Agent Builder
- Part 2: Creating MCP tools using the ADK

What you will learn

Once you have completed this lab you will have developed skills enabling you to complete the following tasks:

- Installing servers requiring environment variables.
- Creating and configuring a connection.
- Import a tool from an MCP server.
- Create a MCP tool withing the ADK.

How to get support

If you encounter an issue with this lab,

- IBM employees can open an IBM Support ticket at <http://ibm.biz/support>.
- Business Partners can request help at the [Partner Plus Support](#) website.

Prerequisites

Make sure the following prerequisites are satisfied before proceeding with this lab:

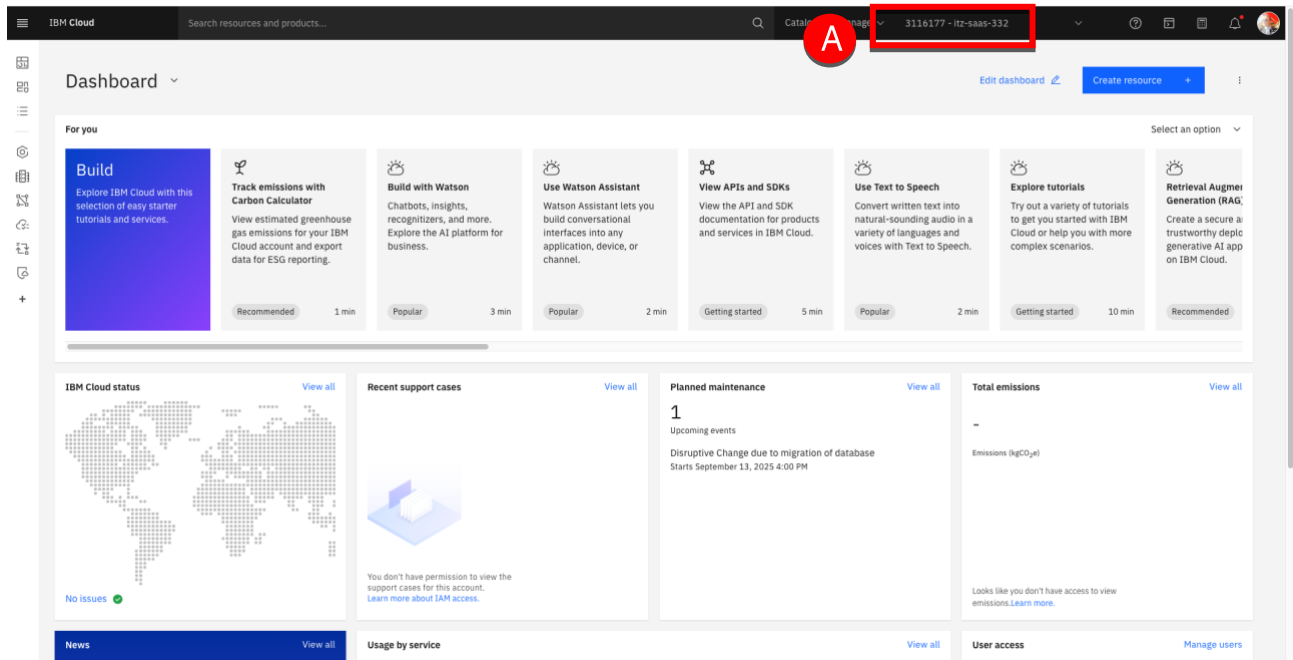
- You have performed the Preparation Lab guide including the environment validation step
- You have your local **ADK 2.12.x** or higher environment up and running (Orchestrate server started after validating the environment in the preparation guide OR after having performed another ADK related lab)

Note: If for some reason your server is not started correctly, reset your server following [the Reset your environment](#) appendix.

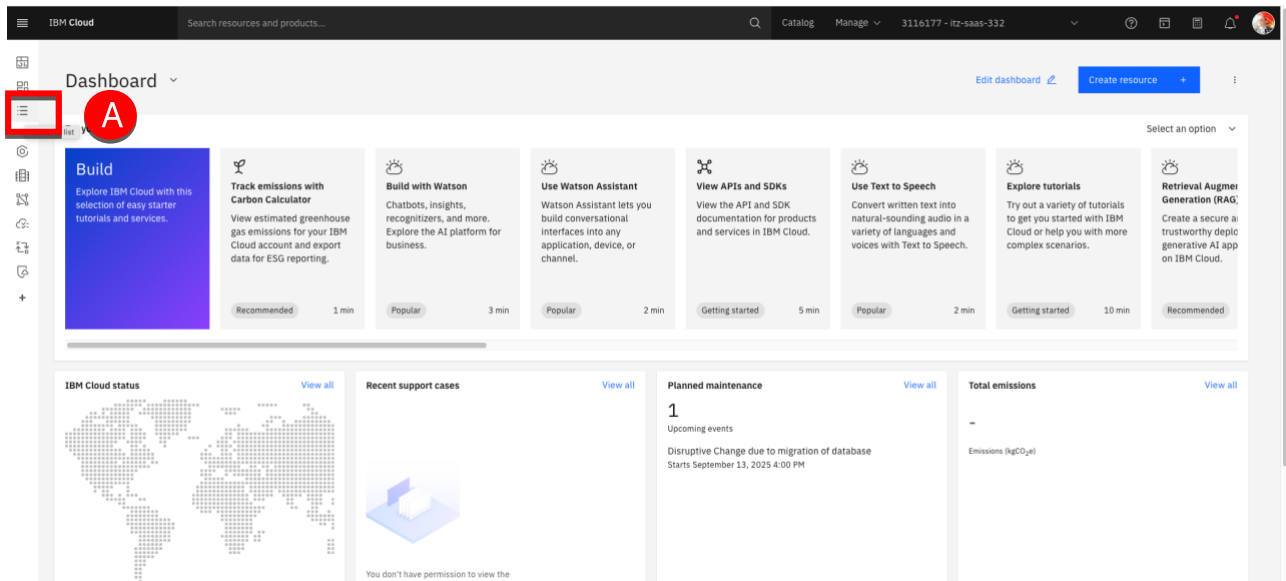
PART 1: Import MCP tools using watsonx Orchestrate

In the first part of this lab, you will learn how to work with MCP tools using watsonx Orchestrate running in IBM Cloud. Refer to your lab credentials in the preparation guide and login to IBM Cloud, ensure you are using the correct account, see item 6 in the lab credentials.

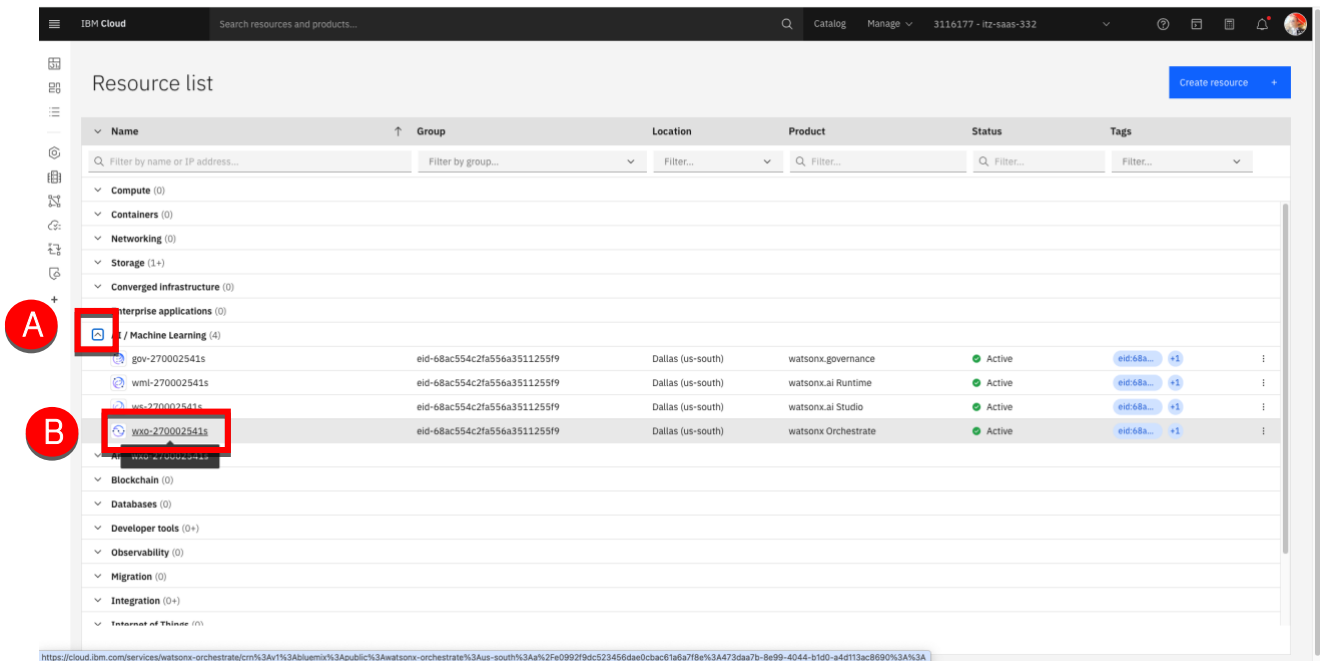
1. Make sure you are on the right cloud account as noted in item 6 in your lab credential notes. This is the IBM Cloud account named in the invitation email you received.



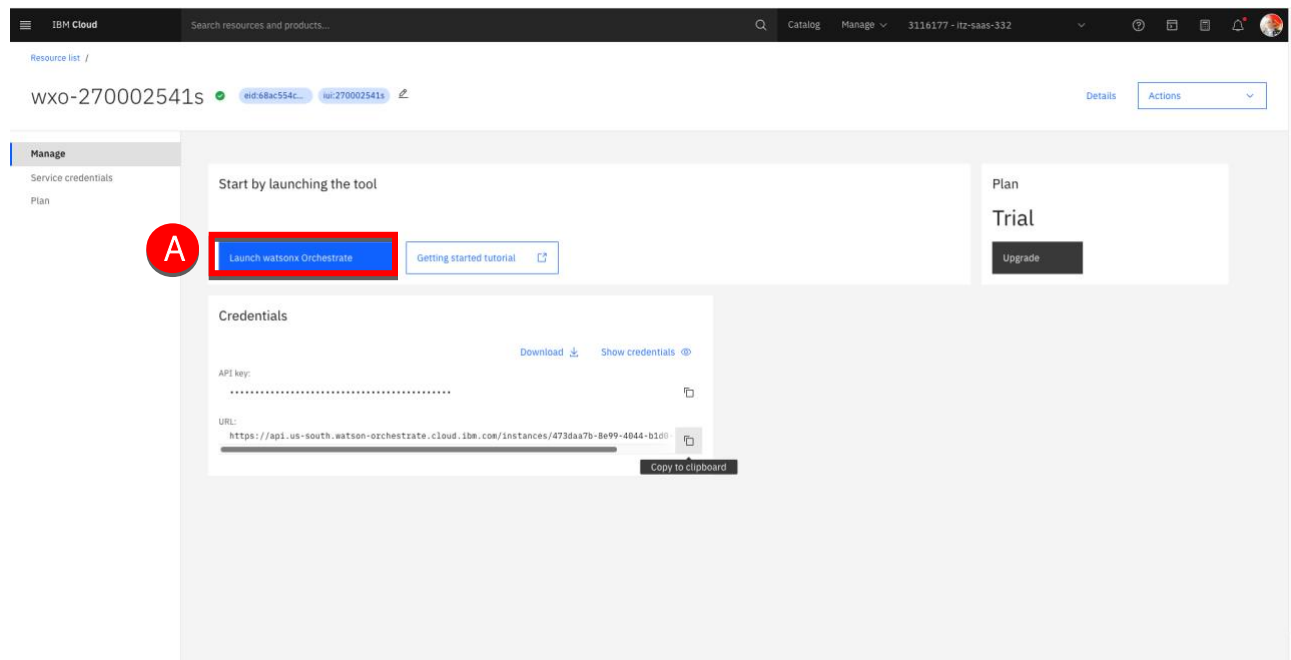
2. Click the Resource list (A).



3. Expand **AI/Machine learning** (A) then click **watsonx Orchestrate** (B).



4. Click **Launch watsonx Orchestrate** (B).



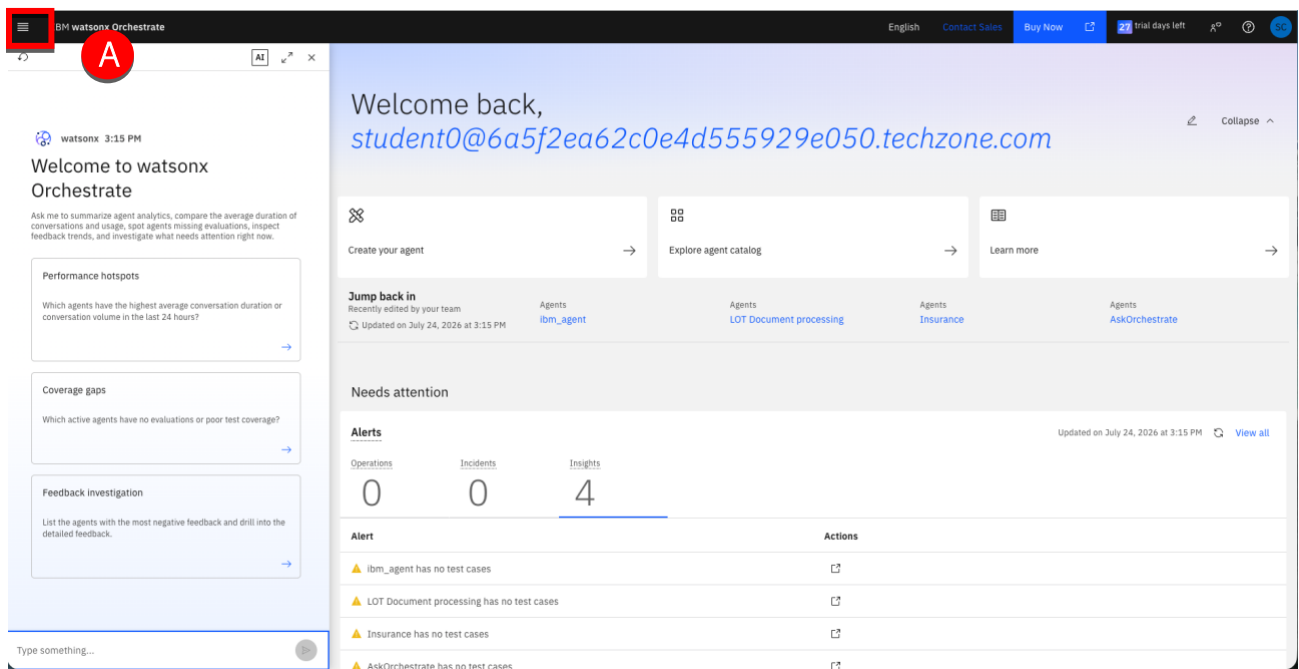
Create a Tavily connection

To install MCP servers that require environment variables, you must add the required variables into an AppID/Connection using key-value pairs. MCP servers fetch the environment variables when they start, so it is important to set the variables before you import or start an MCP server.

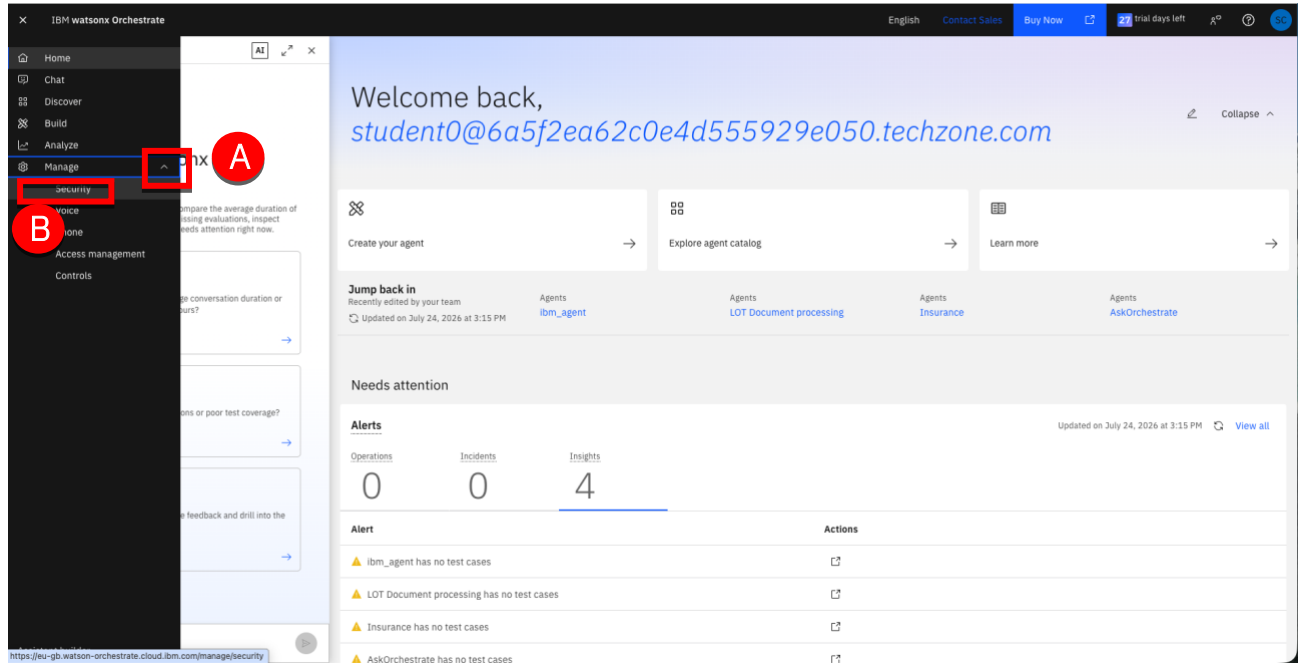
To connect to the Tavily server with your credentials, you must create a new connection in watsonx orchestrate. This is the most secured way to import tools not displaying your API keys and credentials publicly.

Return to your watsonx Orchestrate top menu.

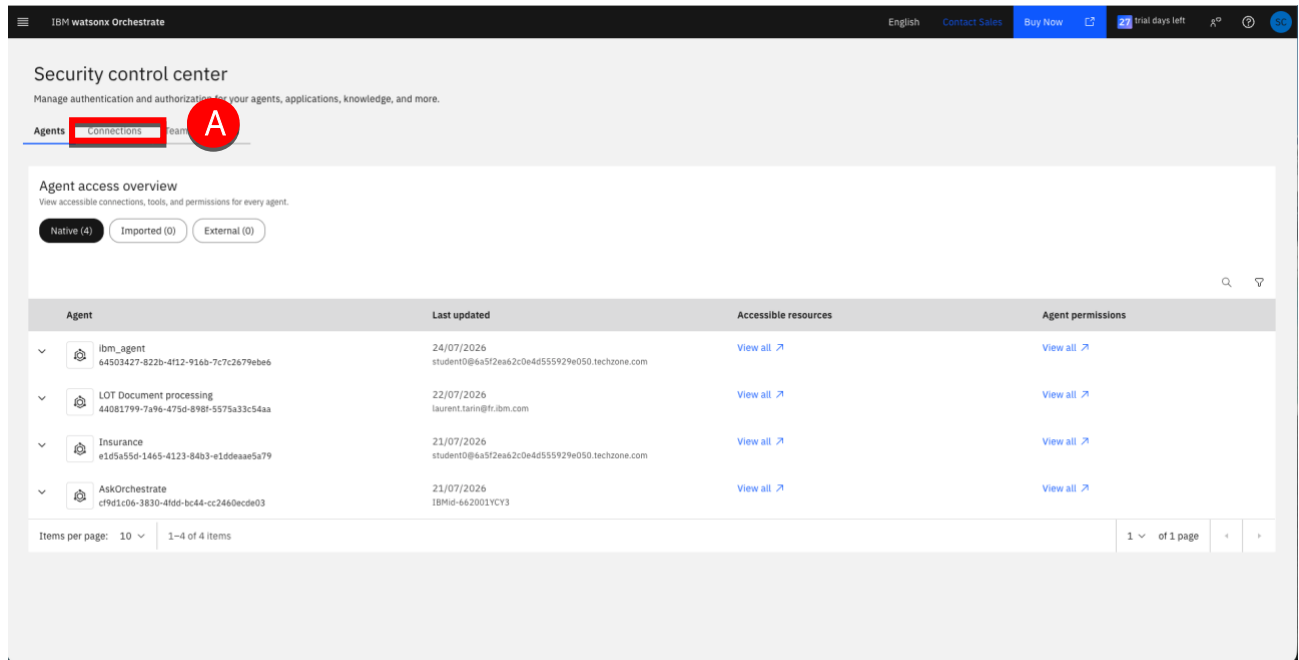
1. Click the Main menu icon (A).



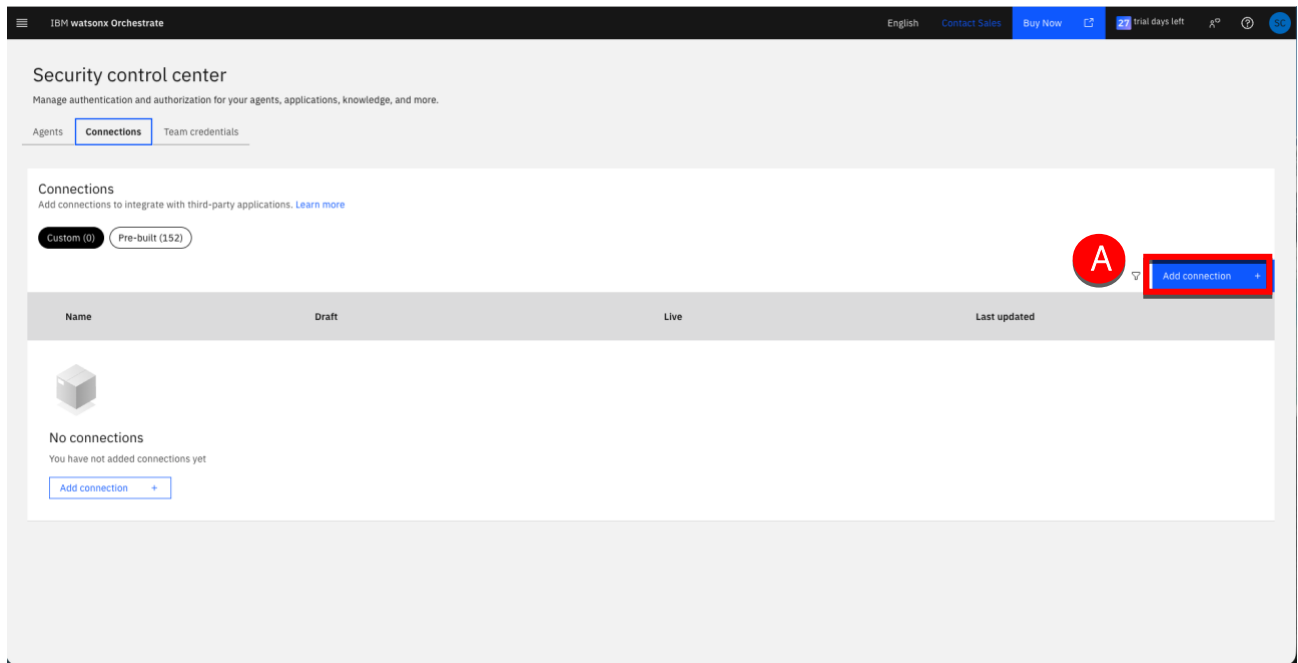
- Expand the **Manage** menu (A) then select **Security** (B).



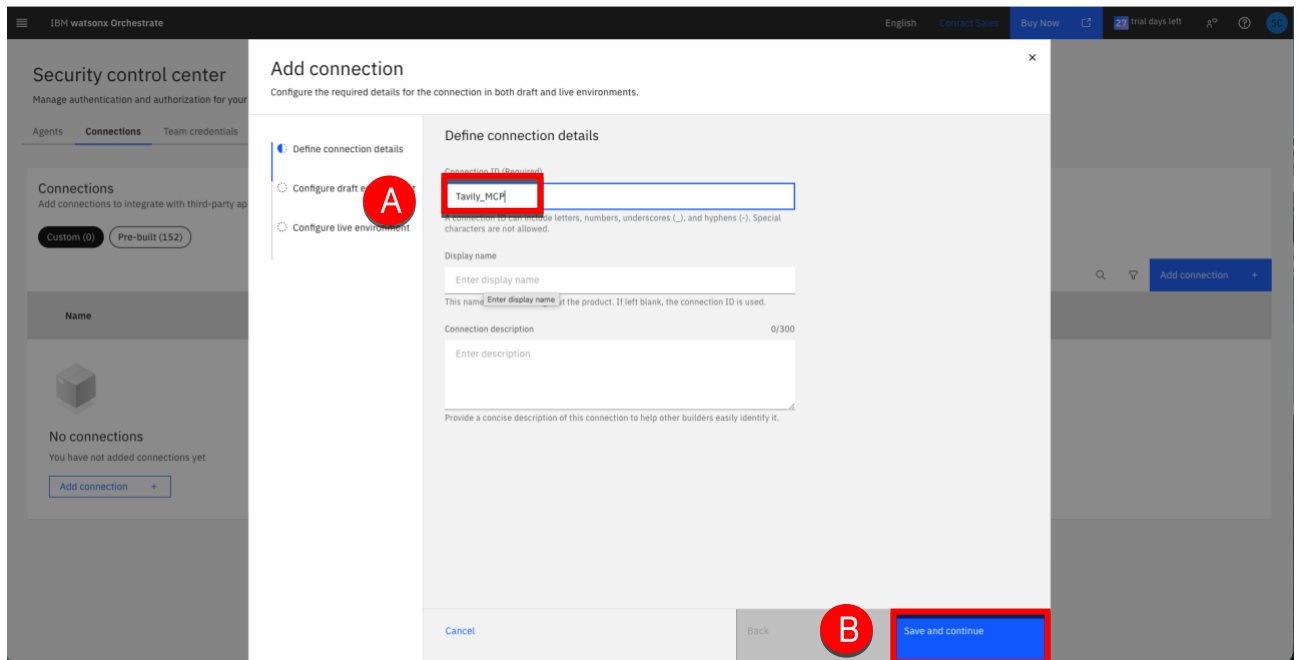
- Click the **Connections** tab (A).



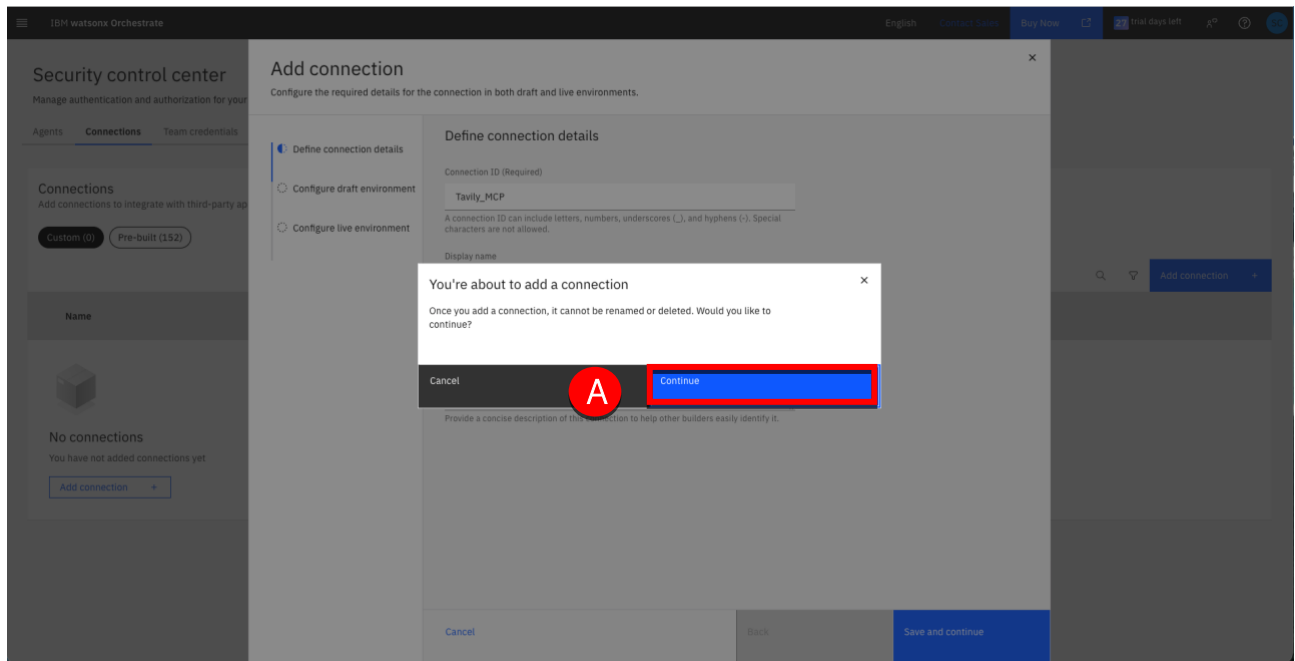
4. Click **Add connection** (A).



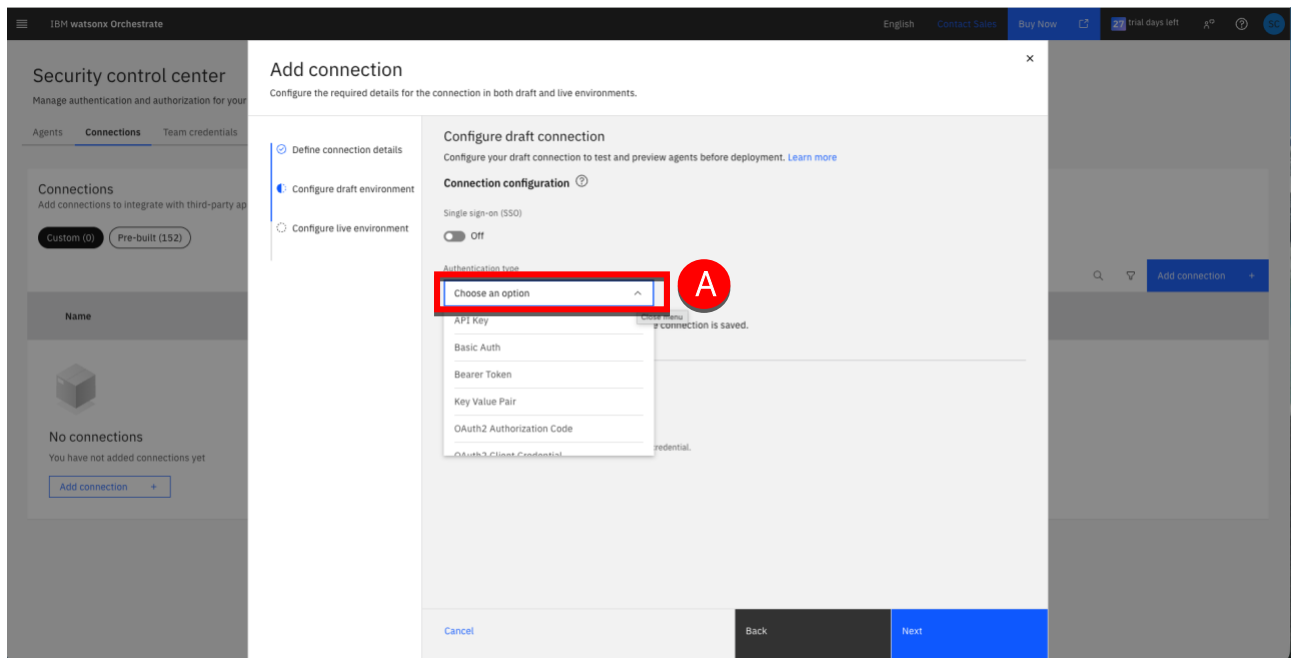
5. Enter **Tavily_MCP** in the **Connection ID*** field (A), click **Save and continue** (B).



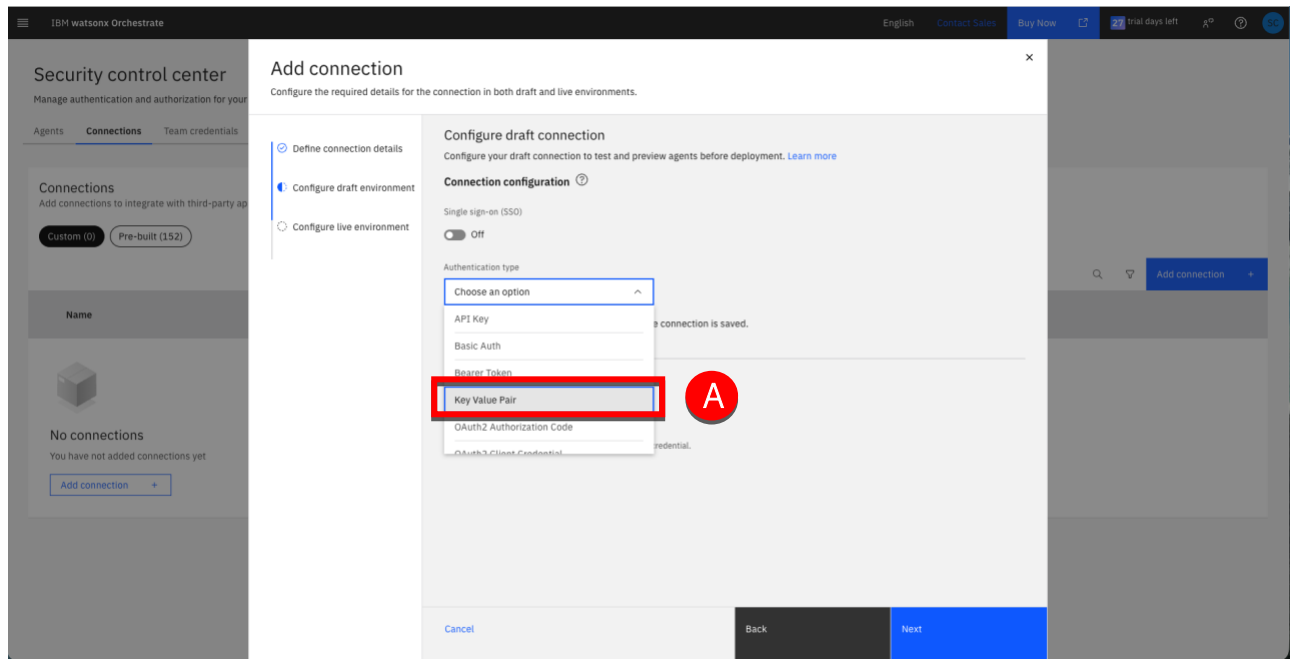
6. Click **Continue** (A).



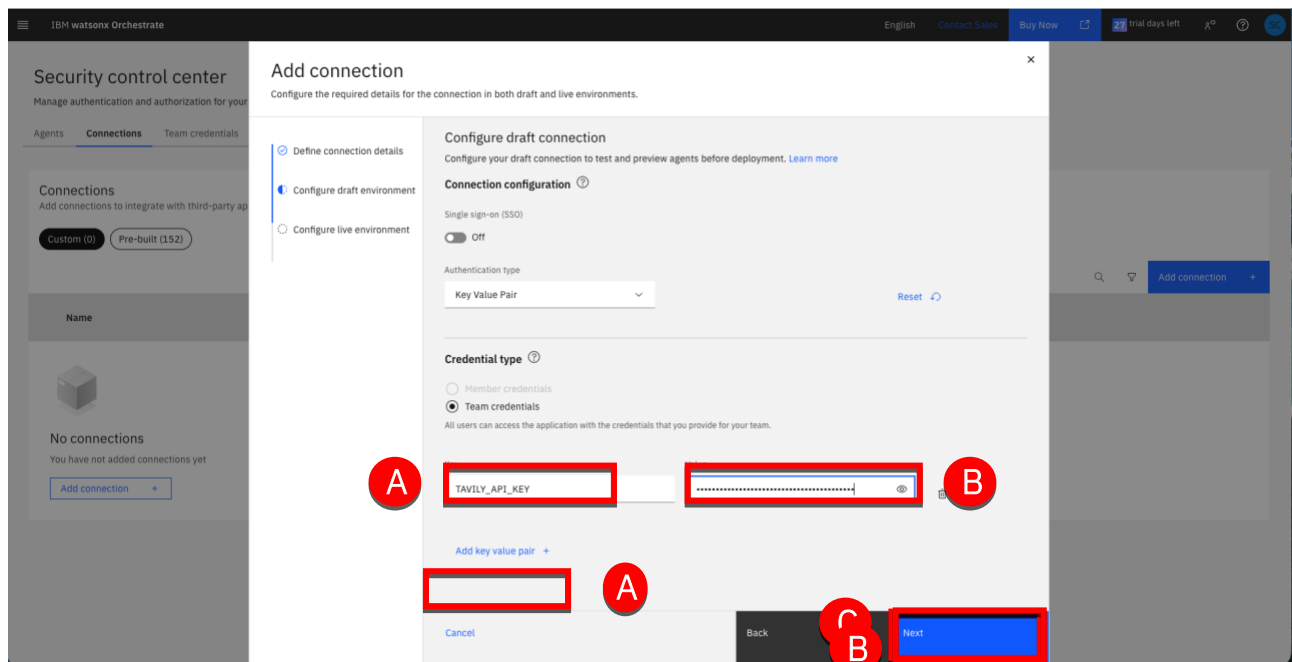
7. Expand the Choose an option in the **Authentication type** menu (A).



8. Select **Key Value Pair** (A).



9. Type **TAVILY_API_KEY** in the **Key** name field (A), copy your Tavily API key from your note in the **Value** field (B), Click **Next** (C).



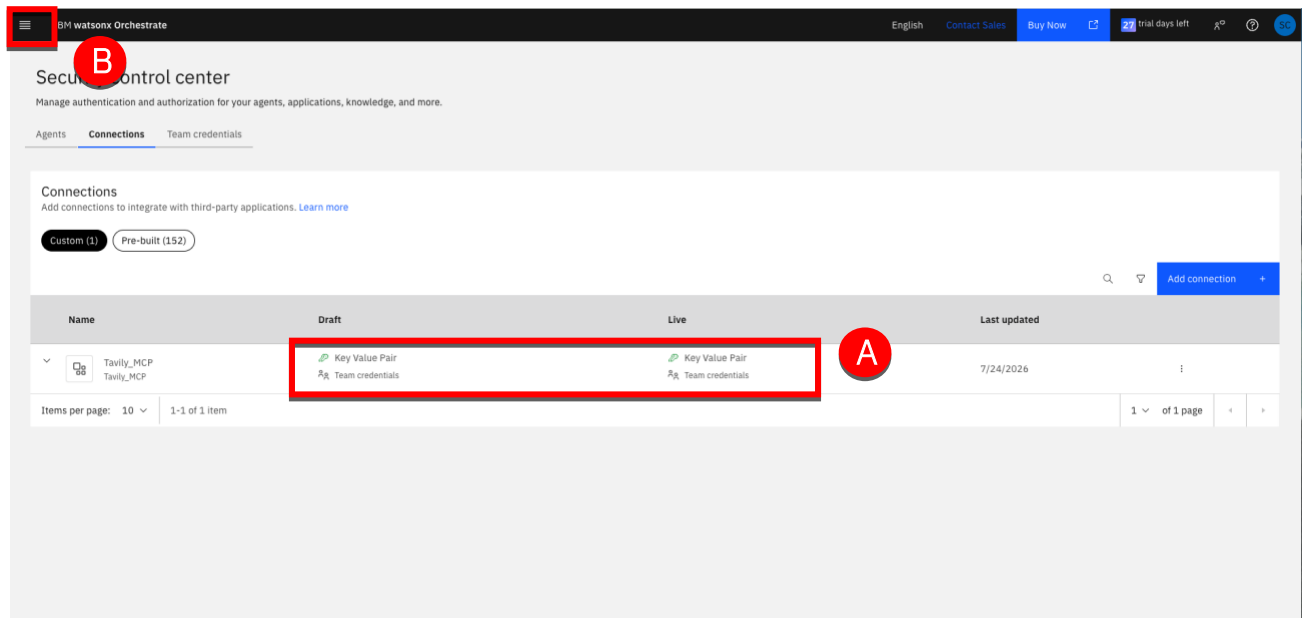
10. Repeat from Step 5 and add the credentials for the **Live** environment.

The screenshot shows the 'Add connection' dialog in the IBM Watsonx Orchestrator. The left sidebar shows the 'Connections' tab with 'Custom (0)' and 'Pre-built (152)' buttons. The main panel is titled 'Add connection' and 'Configure the required details for the connection in both draft and live environments.' It has three steps: 'Define connection details', 'Configure draft environment', and 'Configure live environment'. The 'Configure live environment' step is selected and highlighted with a red box. The 'Configure live connection' section shows 'Connection configuration' with a 'Paste draft configuration' button. Below this, 'Single sign-on (SSO)' is set to 'Off'. 'Authentication type' is set to 'Key Value Pair'. The 'Credential type' section has 'Team credentials' selected. The 'Key' field contains 'TAVILY_API_KEY' and the 'Value' field contains a masked password. A red box highlights the 'Finish' button at the bottom right.

11. Click **Finish** (A) when done.

This screenshot is identical to the previous one, showing the 'Add connection' dialog with the 'Configure live environment' step selected. A red circle with the letter 'A' is placed over the 'Finish' button, which is also highlighted with a red box, indicating the final step to complete the connection setup.

12. Check that your connection is now ready (A), both draft and live environments will have access to your credentials in a secured way. Click the top menu (B).

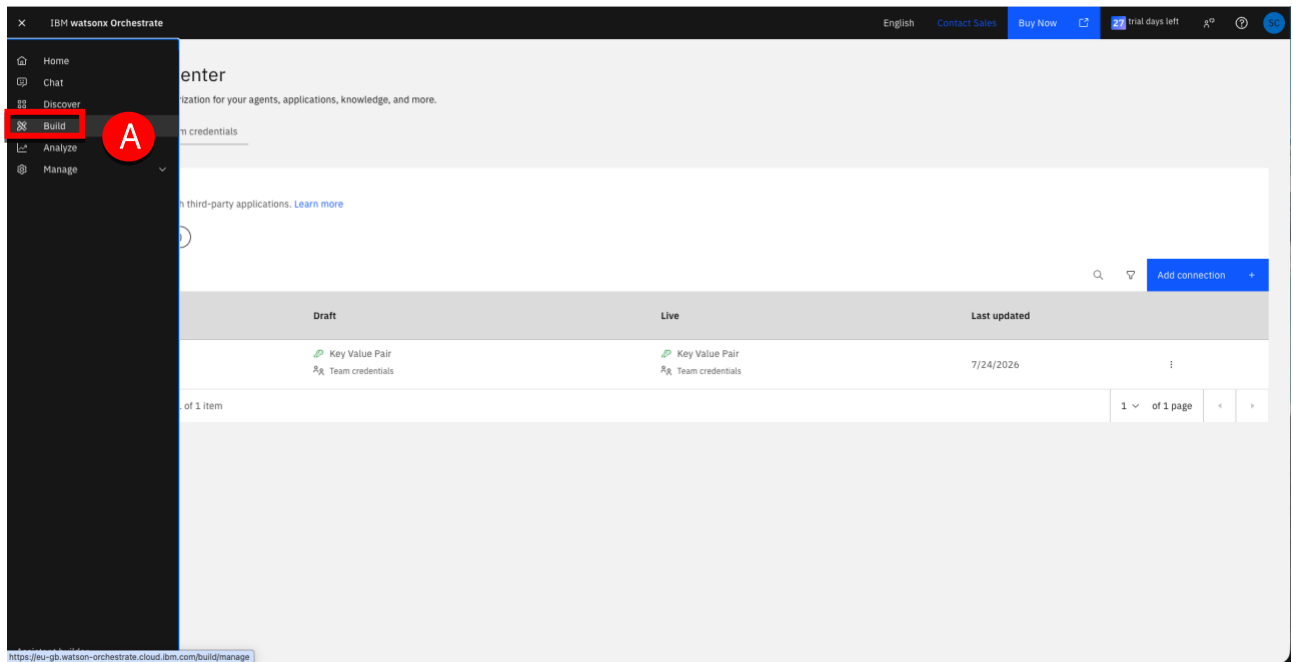


Using Local MCP servers to run tools

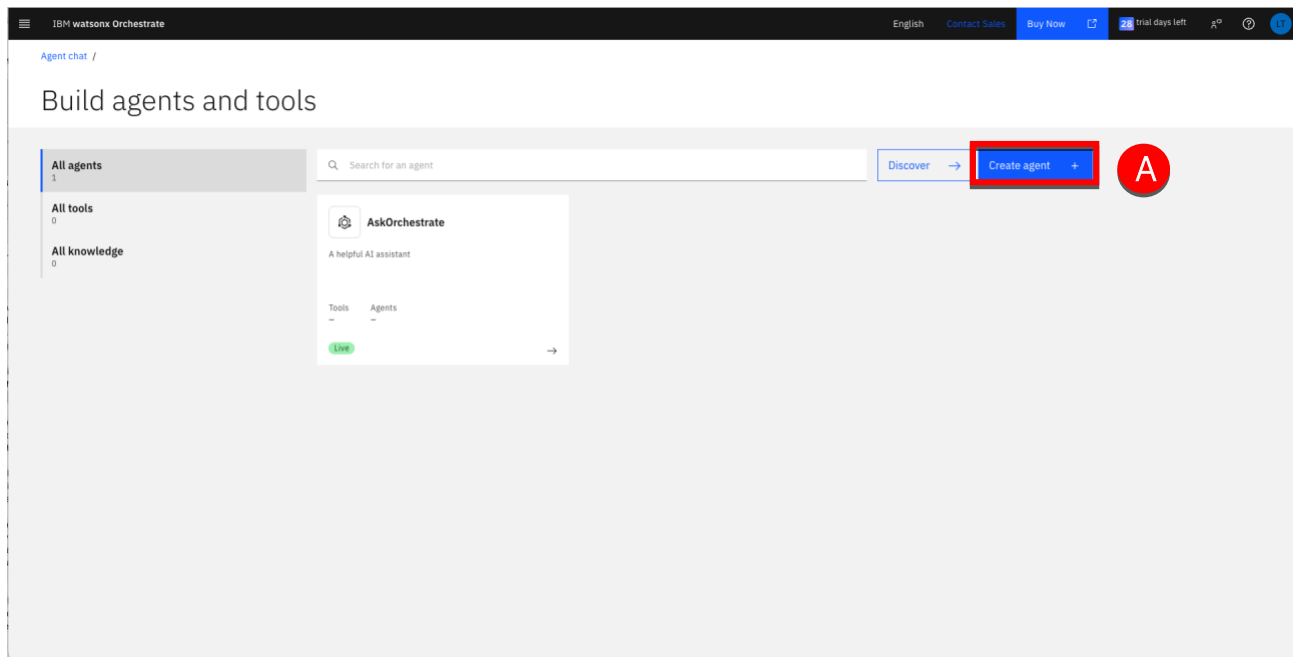
Next, you must create a new agent. This new agent will contain the Tavily search tool you will import from the Tavily MCP server. The connection to the Tavily server will be managed through the connection you have created in the previous step.

Create an agent

1. Click the **Build** menu (A).



2. Click **Create agent** (A).



3. Select **Create from scratch** (A).

IBM watsonx Orchestrate

English Contact Sales Buy Now Trial days left

Create an agent

Create from scratch
Build your custom agent step by step to create a custom solution.

Create from template
Browse the catalog and use the attributes of another agent as a template to create your agent.

Import LangGraph agent Preview

Global wor...

Name * 0/40
Name your agent

Description *
Example: This agent helps answer customer questions about return and cancellation policies.
Describe your agent, including its purpose, tools, and agents, to help other agents and users know when to use it.
What makes a good description?

Cancel Create

4. Type 'Tavily_search_agent' (A) in the **Name** field. Type 'Use this agent to perform advanced web searches using Tavily. This agent will search over the web based on the input query you will provide.' in the **Description** (B). Click **Create** (C).

IBM watsonx Orchestrate

English Contact Sales Buy Now Trial days left

Create an agent

Create from scratch
Build your custom agent step by step to create a custom solution.

Create from template
Browse the catalog and use the attributes of another agent as a template to create your agent.

Import LangGraph agent Preview

Global wor...

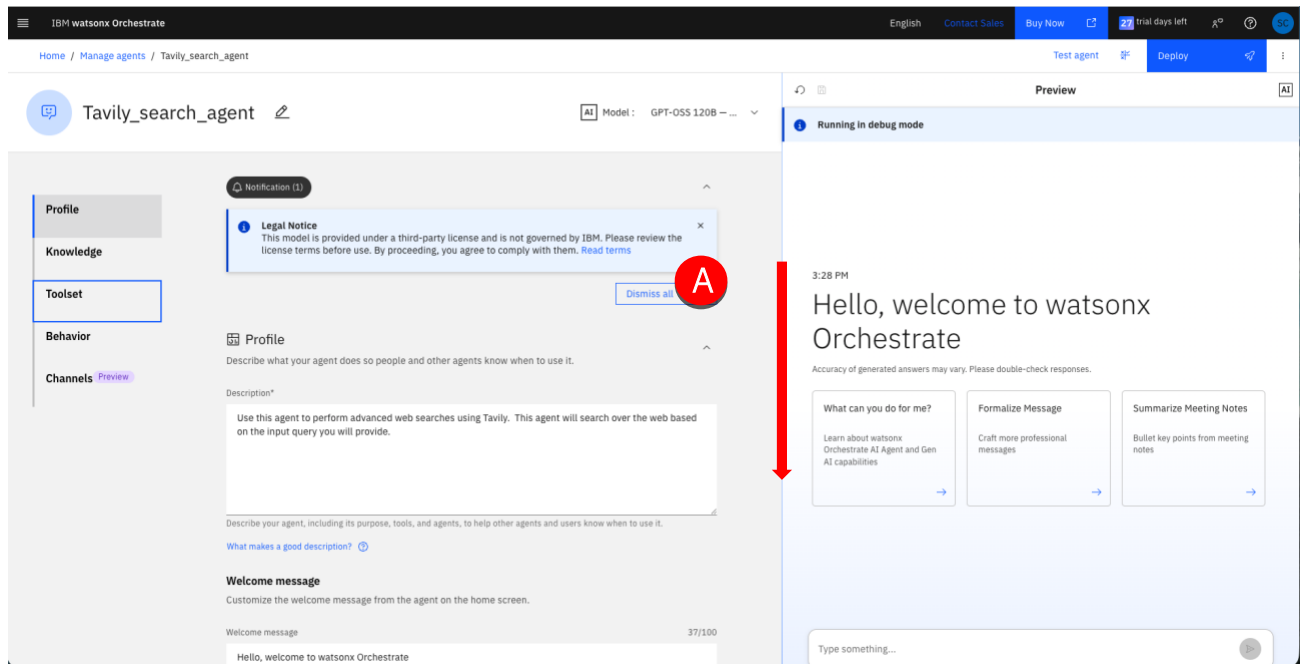
Name * 19/40
Tavily_search_agent

Description *
Use this agent to perform advanced web searches using Tavily. This agent will search over the web based on the input query you will provide.
Describe your agent, including its purpose, tools, and agents, to help other agents and users know when to use it.
What makes a good description?

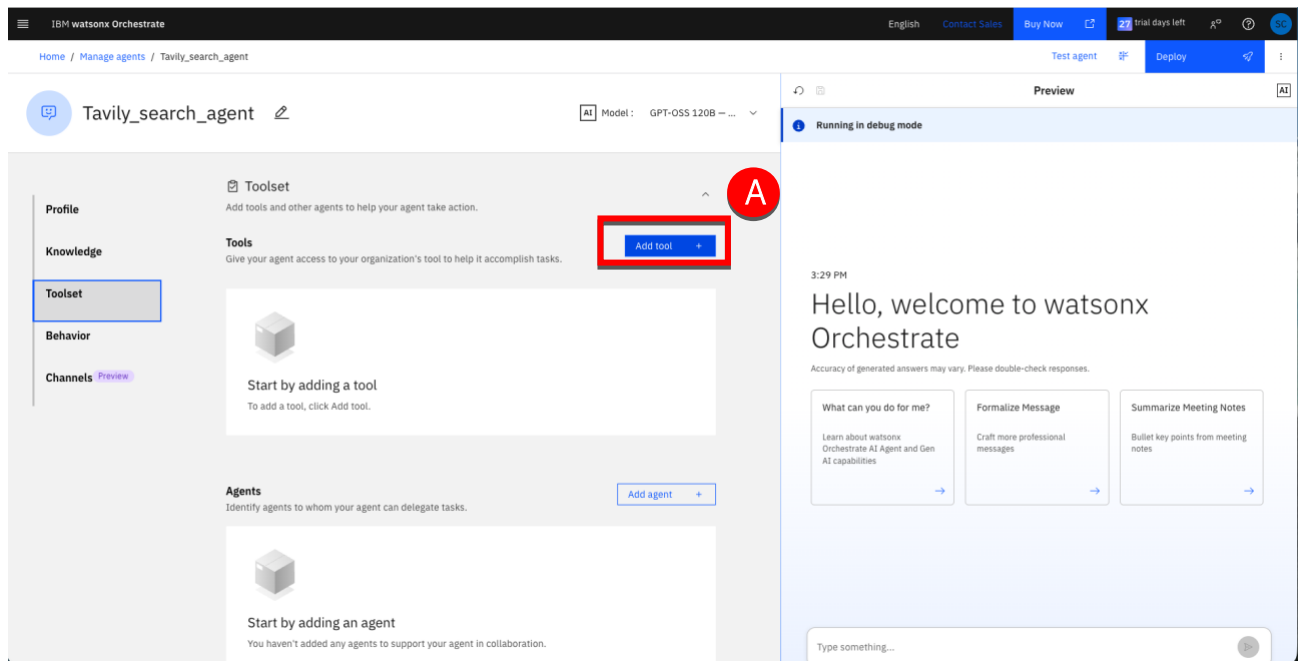
Cancel Create

Configure an MCP Server

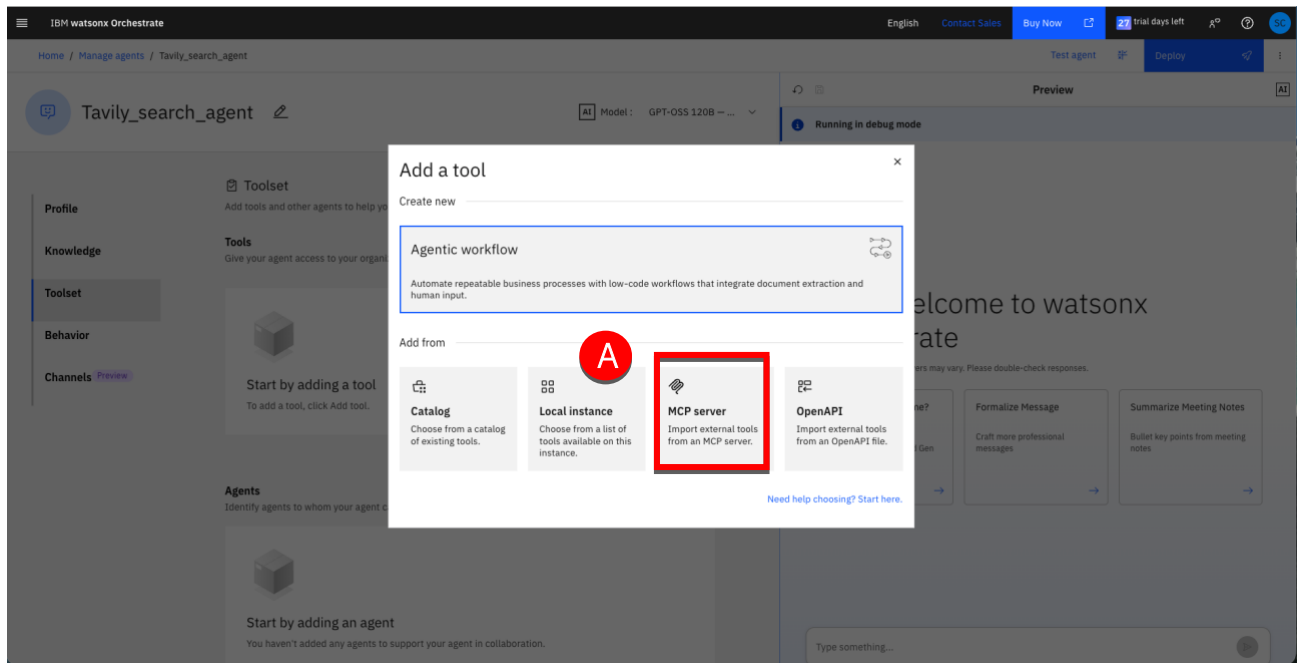
1. Scroll down to the **toolset** section of your agent (A).



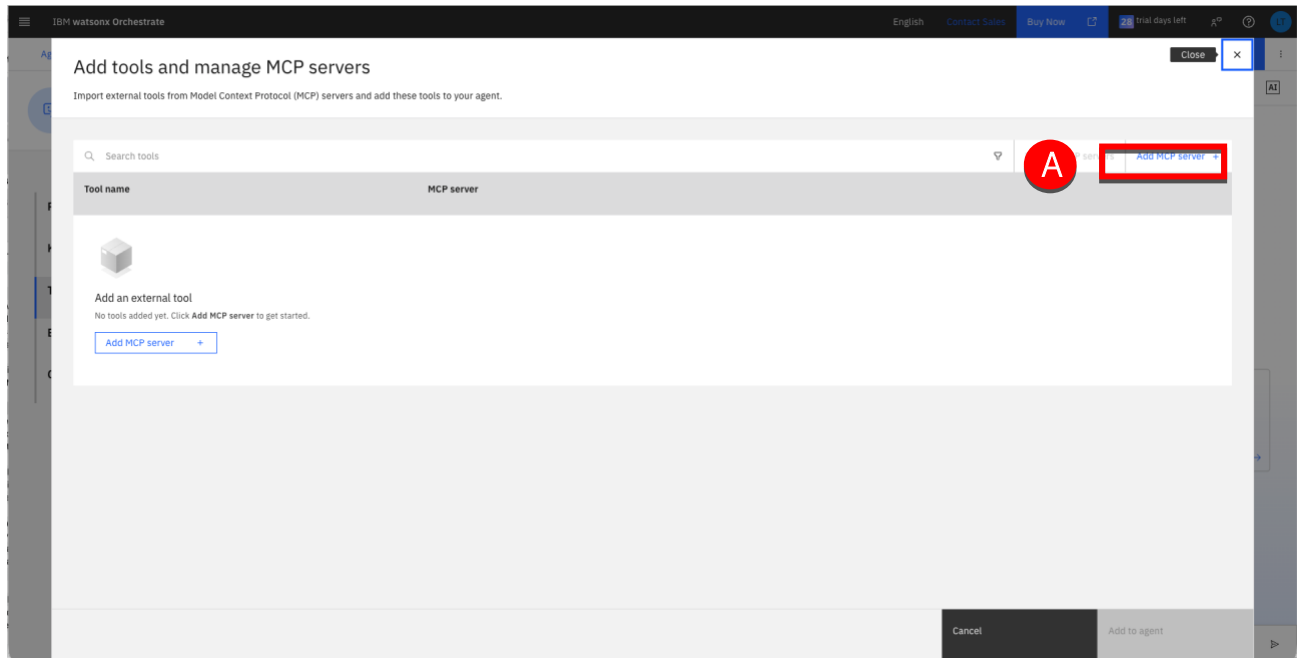
2. Click **Add tool** (A).



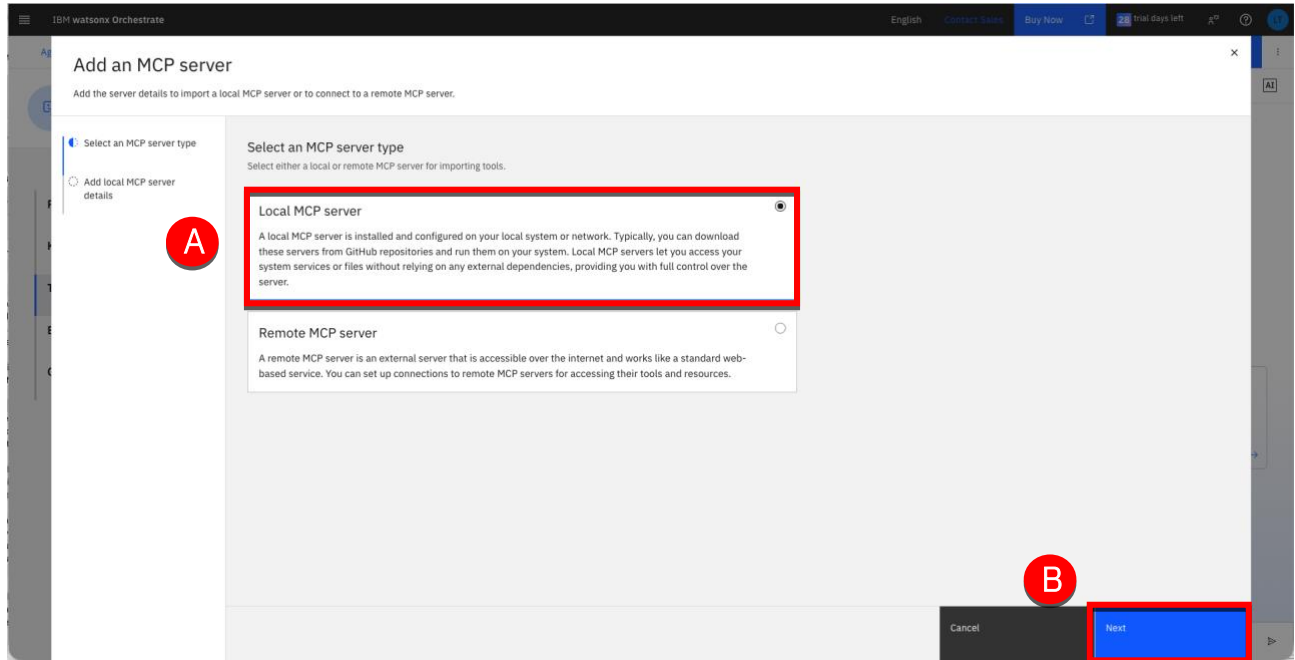
3. Click MCP Server (A).



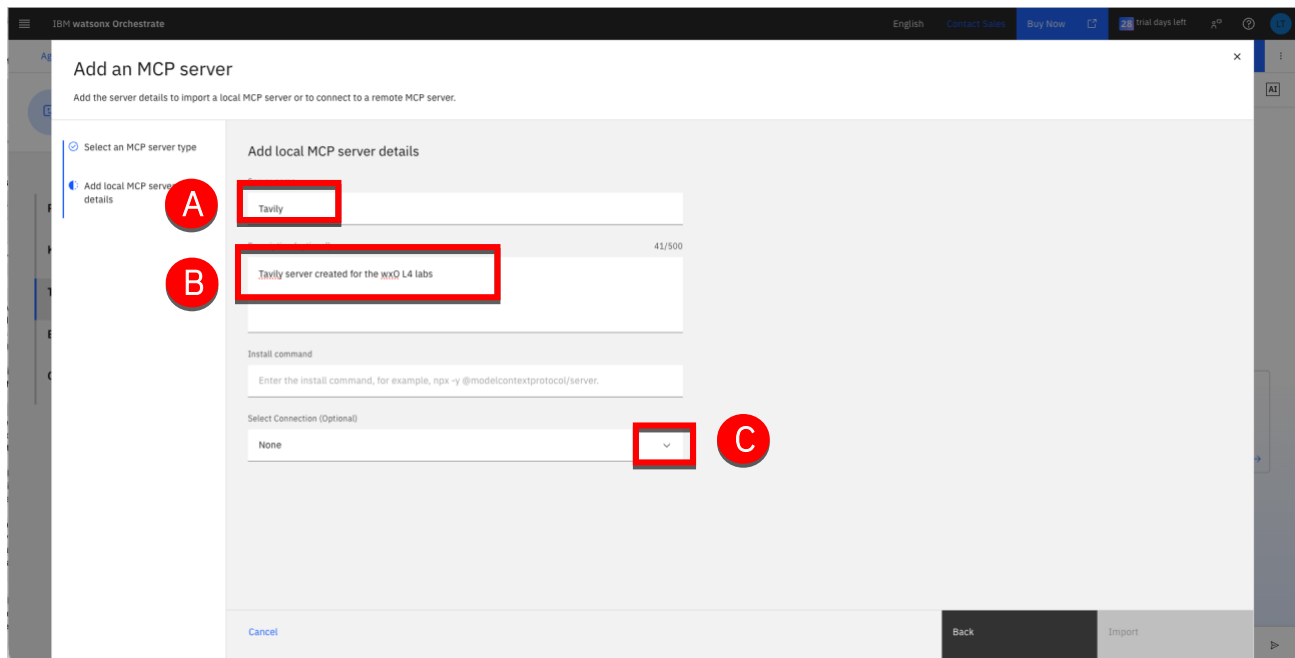
4. Click Add MCP server (A).



5. Select Local MCP server (A), click Next (B).



6. Type 'Tavily' in the **Server name** (A), type 'Tavily server created for the wx0 L4 labs' in the **Description** (B), expand the **Select Connection** drop down (C).



7. Select your **Tavily_MCP** from the list (A).

IBM watsonx Orchestrate

English Contact Sales Buy Now 28 trial days left

Add an MCP server

Add the server details to import a local MCP server or to connect to a remote MCP server.

Select an MCP server type

- Select an MCP server type
- Add local MCP server details

Add local MCP server details

Server name

Tavily

Description (optional) 41/500

Tavily server created for the wxO L4 labs

Install command

Enter the install command, for example, `npx -y @modelcontextprotocol/server`.

Select Connection (Optional)

- Tavily_MCP
- BI Worldwide
- TestRail
- Zendesk
- Maximo
- Red Hat Ansible
- SAP ARIBA Master Data Retrieval
- Dynamics 365
- Uptok
- Spark Compass
- DataVault
- ZoomInfo
- Zendesk

Back Import

8. Type `'npx -y tavily-mcp@0.1.3'` in the command field (A) then click **Import** (B).

IBM watsonx Orchestrate

English Contact Sales Buy Now 28 trial days left

Add an MCP server

Add the server details to import a local MCP server or to connect to a remote MCP server.

Select an MCP server type

- Select an MCP server type
- Add local MCP server details

Add local MCP server details

Server name

Tavily

Description (optional) 41/500

Tavily server created for the wxO L4 labs

Install command

npx -y tavily-mcp@0.1.3

Select Connection (Optional)

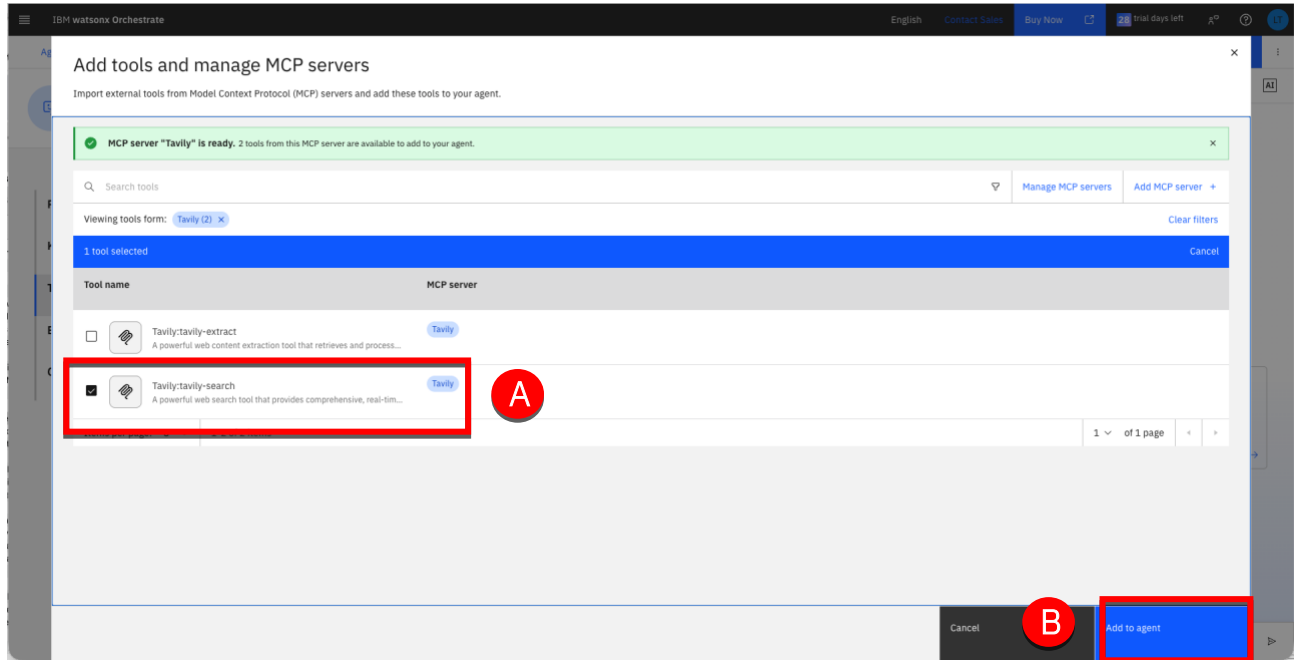
Tavily_MCP

Cancel Back Import

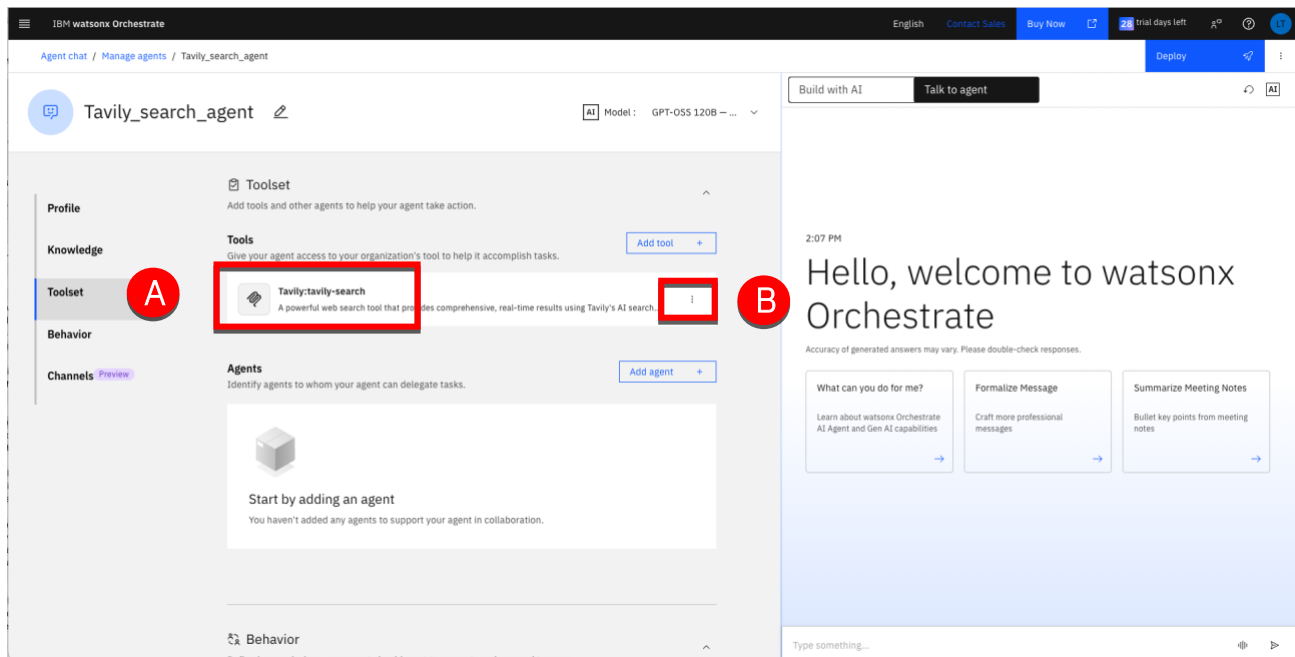
Select the tools to import from the MCP server

Now that your server is imported and you are connected to the server, you can add the tools that are exposed to your agent.

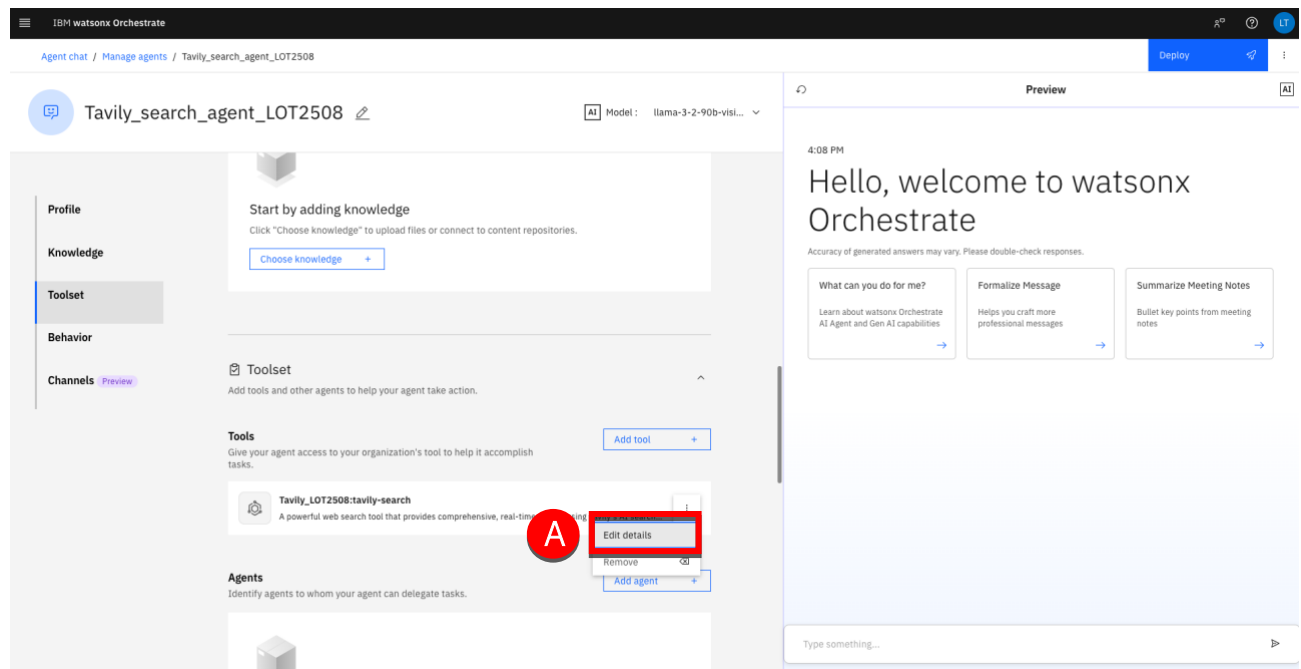
1. Identify the `tavily_search` tool (A) then click **Add to agent** (B).



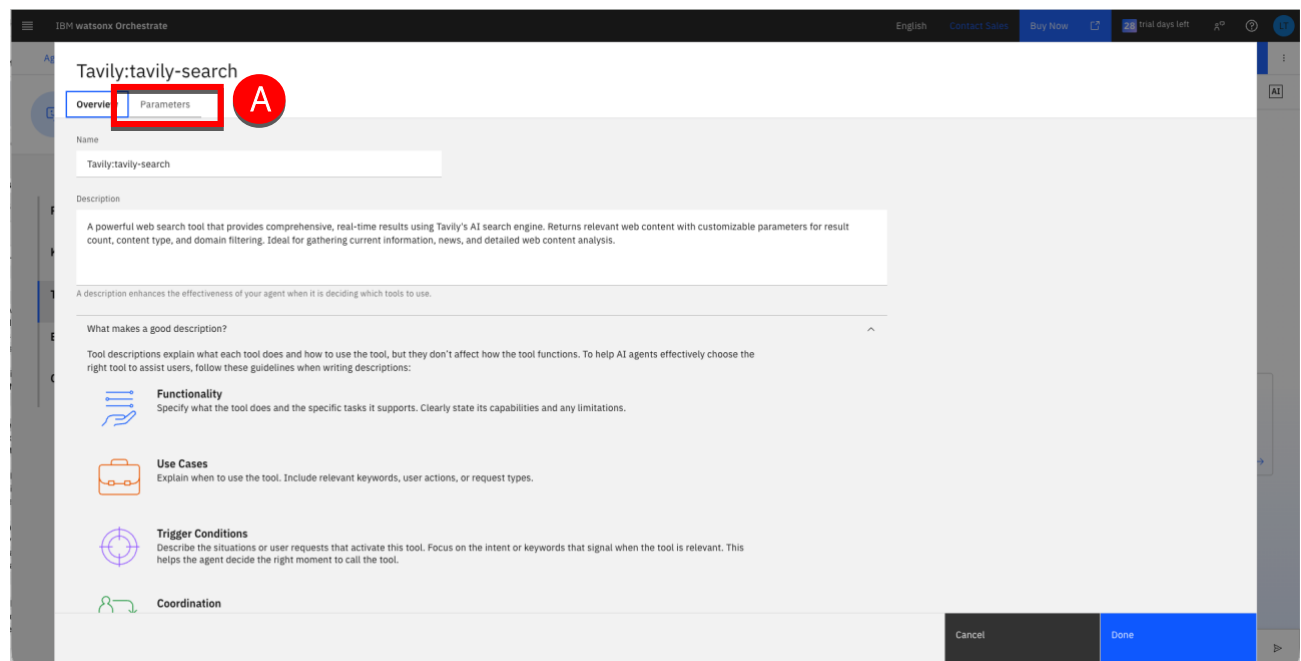
2. Note that the Tavily search tool is now added to your own tools (A). Tavily is an advanced web search tool. More parameters can be specified in addition to the default search query. Click the option menu icon (B).



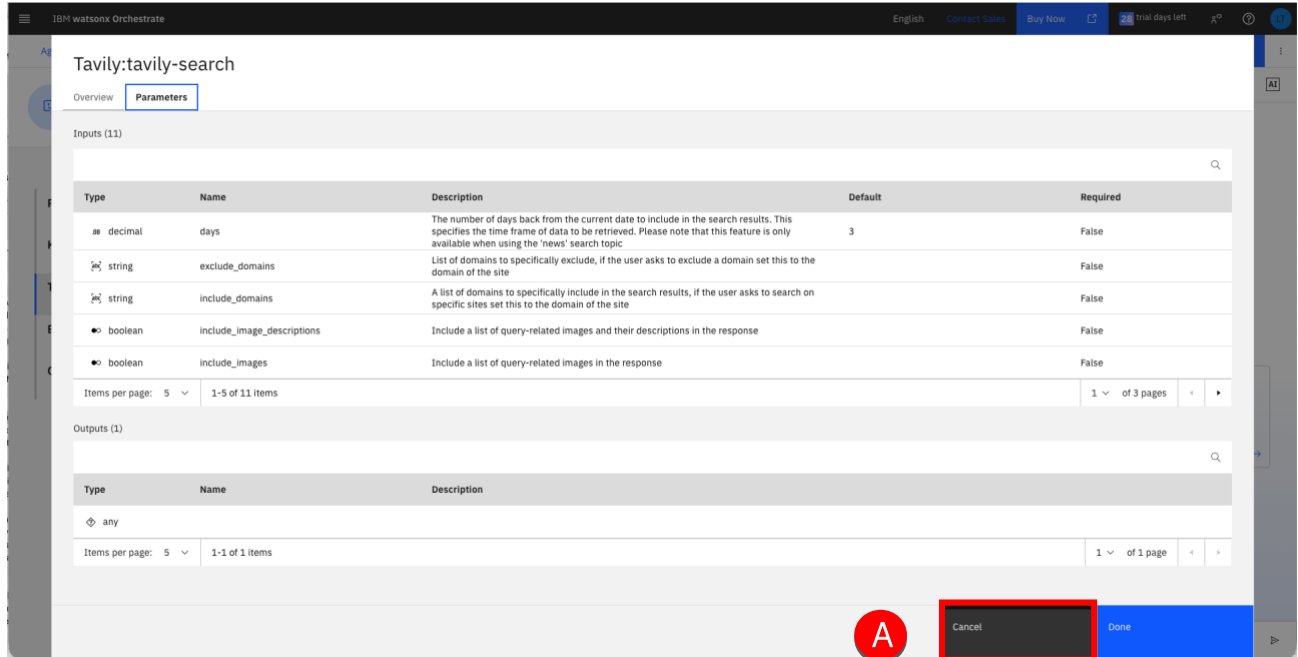
3. Click **Edit details** (A).



4. Click **Parameters** (A).



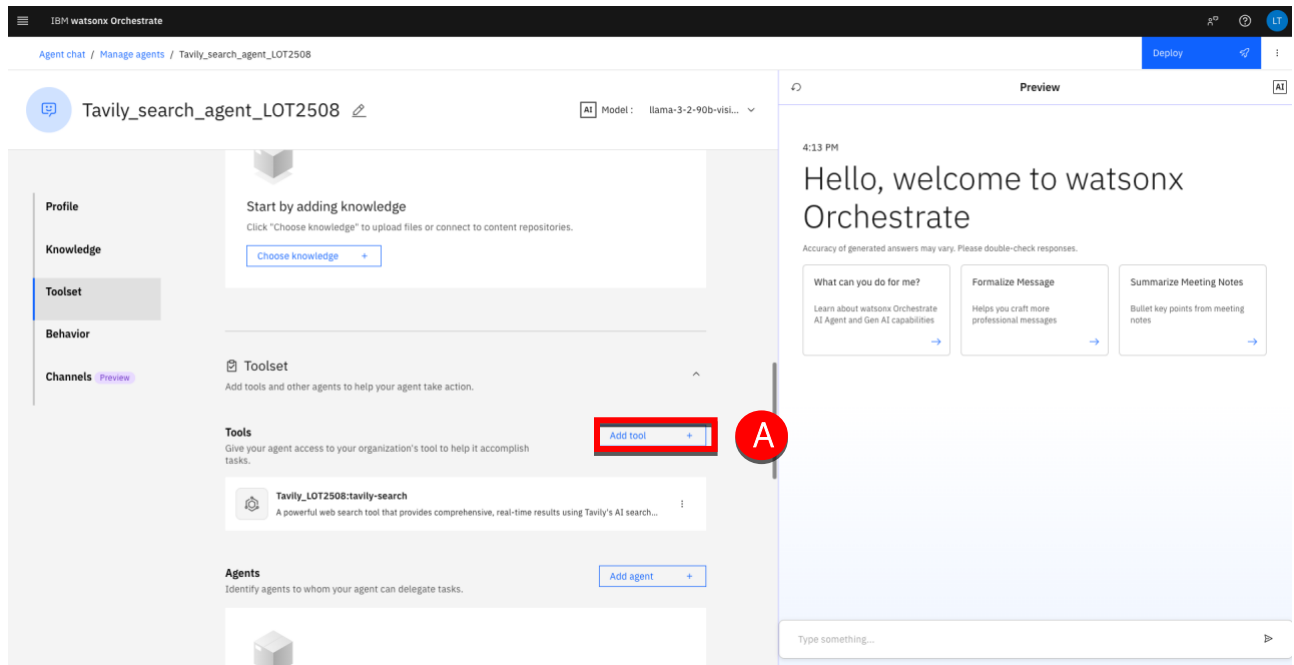
5. By default, only the search query is required to launch the tool. But you can activate more parameters to specify the topic of the search or limit the search to the recent content specifying the number of days since the retrieved content was published. Let's keep with the default query for now. Click **Cancel** (A).



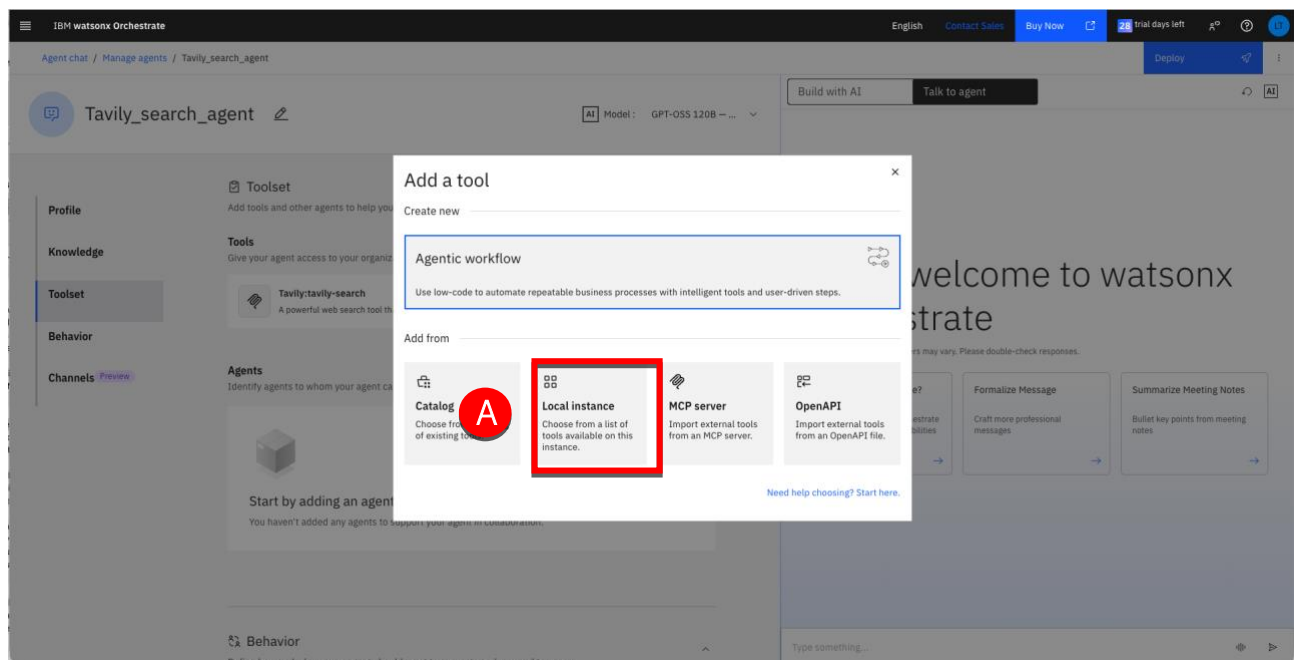
[Adding more tools from your local MCP servers.](#)

MCP servers are created just once when adding the first tool in your watsonx Orchestrate instance. Once the specified import command executed, you can add more tools (if any) from this MCP server. Let's see how.

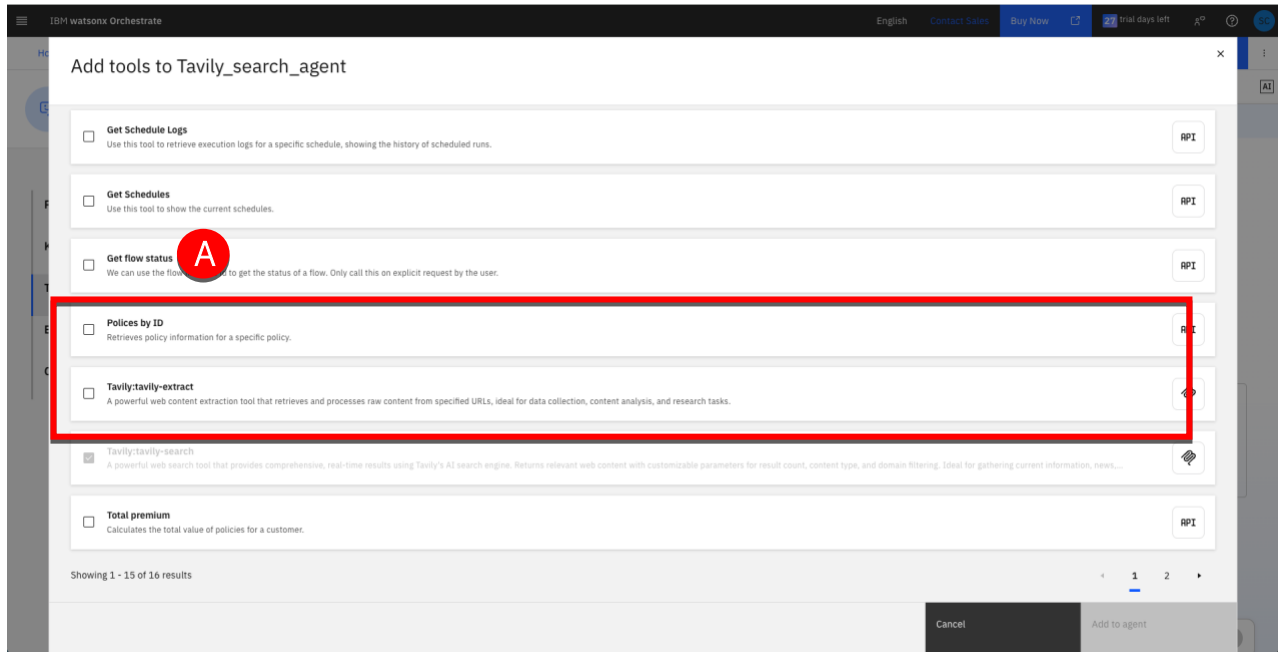
1. Click **Add tool** (A).



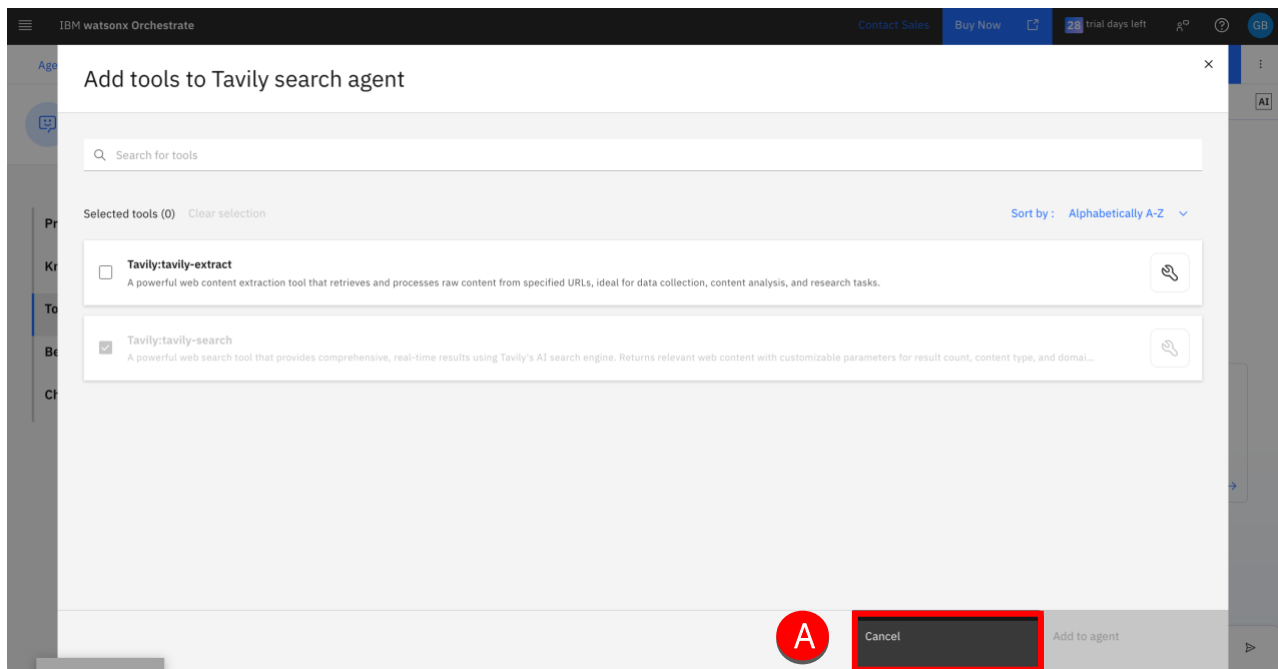
2. Select **Add from a local instance** (A).



3. If you have created other tools they will be listed here (A). If you are using a new tenant, only the Tavily tools will be listed.



4. The Tavily server exposes 2 tools: The Tavily-extract and Tavily-search. The search that you have already added to your agent is grayed out. From here, you can decide to add more tools at any time. We will just use the Search tool in this lab. Click **Cancel** (A).



Preview the tool within your agent

Your tool is now ready to be used in your agent. Let's use watsonx Orchestrate preview to play with it.

1. Type 'What is an MCP server?' in the chat area (A).

IBM watsonx Orchestrate

Agent chat / Manage agents / Tavily_search_agent

Tavily_search_agent

Model: GPT-OSS 120B

Build with AI | Talk to agent

2:11 PM

Hello, welcome to watsonx Orchestrate

Accuracy of generated answers may vary. Please double-check responses.

What can you do for me? Formalize Message Summarize Meeting Notes

what is MCP server?

2. Expand Show reasoning (A).

IBM watsonx Orchestrate

Agent chat / Manage agents / Tavily_search_agent

Tavily_search_agent

Model: GPT-OSS 120B

Build with AI | Talk to agent

2:12 PM

You 2:12 PM

what is MCP server?

Tavily_search_agent 2:12 PM

Show Reasoning

MCP Server – a quick overview

Aspect	What it means
Full name	Model Context Protocol server
Core idea	A small web service that exposes tools, data, or functions (e.g., a database query, an API call, a calculator) in a standardised way so that an AI model (ChatGPT, Claude, Gemini, etc.) can invoke them at runtime.
Why it exists	Large language models are great at reasoning with text, but they cannot directly read your private databases or trigger your internal APIs. MCP provides a common "bridge" that lets an LLM talk to the outside world without custom code for each integration.
How it works (simplified)	1. Define a "tool" – a description of the function (name, parameters, description). 2. Deploy the MCP server – a lightweight HTTP-based service that registers those tool definitions and implements the actual logic. 3. AI client (e.g., Claude with "skills", a custom agent, or any MCP-compatible client) queries the server for its tool list, then calls a tool with JSON arguments. 4. The server runs the code, returns the result, and the AI continues its reasoning.
Key traits	• Standard protocol – JSON-over-HTTP, versioned spec (so any client that knows the spec can talk to any server). • Security-first – you run the server inside your own network or cloud, control authentication, rate limits, and data-privacy. • Extensible – you can expose anything from simple math to complex business-logic micro-services.
Typical use-cases	• Enterprise agents that need to read a CRM, update a ticket, or fetch a sales forecast. • Data-centric AI that wants to query a SQL warehouse or a vector store on-the-fly. • Tool-augmented chatbots that can book meetings, run code, or trigger CI.

Type something...

Note: if your agent replies without invoking the MCP tool (no 'Show reasoning' button), click on the **Instruction** section of your agent to add "use your Tavily too first".

3. Expand Step 1 (A).

The screenshot shows the IBM watsonx Orchestrate interface. On the left, the 'Tavily_search_agent' configuration page is visible, with the 'Toolset' tab selected. The 'Tools' section lists 'Tavily:tavily-search'. On the right, a chat window shows a user query 'what is MCP server?'. The agent's response is expanded to 'Step 1', showing a red box around the 'Step 1' label and a red circle with the letter 'A' next to the tool invocation. Below the chat, a table provides a detailed overview of the MCP Server.

Aspect	What it means
Full name	Model Context Protocol server
Core idea	A small web service that exposes tools, data, or functions (e.g., a database query, an API call, a calculator) in a standardised way so that an AI model (ChatGPT, Claude, Gemini, etc.) can invoke them at runtime.
Why it exists	Large language models are great at reasoning with text, but they cannot directly read your private databases or trigger your internal APIs. MCP provides a common "bridge" that lets an LLM talk to the outside world without custom code for each integration.
How it works (simplified)	1. Define a "tool" – a description of the function (name, parameters, description). 2. Deploy the MCP server – a lightweight HTTP-based service that registers those tool definitions and implements the actual logic. 3. AI client (e.g., Claude with "skills", a custom agent, or any MCP-compatible client) queries the server for its tool list, then calls a tool with JSON arguments. 4. The server runs the code, returns the result, and the AI continues its reasoning.
Key traits	• Standard protocol – JSON-over-HTTP, versioned spec (so any client that knows the spec can talk to any server). • Security-first – you run the server inside your own network or cloud, control authentication, rate limits, and data-privacy. • Extensible – you can expose anything from simple math to complex business-logic micro-services. • Enterprise agents that need to read a CRM, update a ticket, or fetch a sales.

4. Note the query the agent extracted from your input query and how it was used to invoke the Tavily search engine (A). By default, the topic was set to 'General' as haven't specified this input parameter as required in the tool settings in the previous steps.

The screenshot shows the IBM watsonx Orchestrate interface, similar to the previous one. The chat window shows the user query 'what is MCP server?'. The agent's response is expanded to 'Step 1', showing a red box around the 'Step 1' label and a red circle with the letter 'A' next to the tool invocation. Below the chat, a table provides a detailed overview of the MCP Server.

Aspect	What it means
Full name	Model Context Protocol server

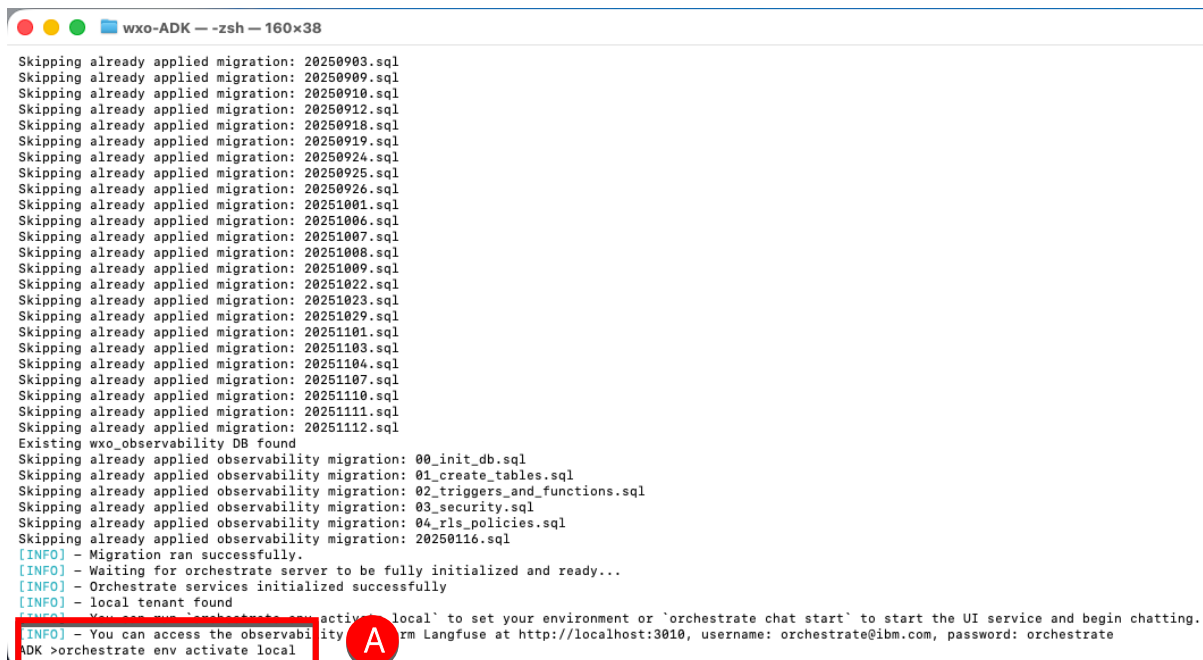
PART 2: Import MCP tools using watsonx Orchestrate Developer Edition (ADK)

In this first part, you will learn how to work with MCP tools using the watsonx Orchestrate Developer Edition using the ADK commands.

Open your ADK environment

1. Open your ADK terminal as described in the preparation guide. Refer to the [Reset your environment appendix](#) if your server is not started.
2. Enter the following command to make sure you are running on the local environment (A).

```
orchestrate env activate local
```



```
wxo-ADK --zsh -- 160x38
Skipping already applied migration: 20250903.sql
Skipping already applied migration: 20250909.sql
Skipping already applied migration: 20250910.sql
Skipping already applied migration: 20250912.sql
Skipping already applied migration: 20250918.sql
Skipping already applied migration: 20250919.sql
Skipping already applied migration: 20250924.sql
Skipping already applied migration: 20250925.sql
Skipping already applied migration: 20250926.sql
Skipping already applied migration: 20251001.sql
Skipping already applied migration: 20251006.sql
Skipping already applied migration: 20251007.sql
Skipping already applied migration: 20251008.sql
Skipping already applied migration: 20251009.sql
Skipping already applied migration: 20251022.sql
Skipping already applied migration: 20251023.sql
Skipping already applied migration: 20251029.sql
Skipping already applied migration: 20251101.sql
Skipping already applied migration: 20251103.sql
Skipping already applied migration: 20251104.sql
Skipping already applied migration: 20251107.sql
Skipping already applied migration: 20251110.sql
Skipping already applied migration: 20251111.sql
Skipping already applied migration: 20251112.sql
Existing wxo_observability DB found
Skipping already applied observability migration: 00_init_db.sql
Skipping already applied observability migration: 01_create_tables.sql
Skipping already applied observability migration: 02_triggers_and_functions.sql
Skipping already applied observability migration: 03_security.sql
Skipping already applied observability migration: 04_rls_policies.sql
Skipping already applied observability migration: 20250116.sql
[INFO] - Migration ran successfully.
[INFO] - Waiting for orchestrate server to be fully initialized and ready...
[INFO] - Orchestrate services initialized successfully
[INFO] - local tenant found
[INFO] - You can access the observability UI from Langfuse at http://localhost:3010, username: orchestrate@ibm.com, password: orchestrate
ADK > orchestrate env activate local
```

Create a Tavily connection

Like you have performed in part 1, a Tavily connection must be created to connect watsonx Orchestrate to the Tavily server you are using.

Create the connection

1. Type the following command to create a connection with an application id called tavily

```
orchestrate connections add --app-id Tavily
```

```
Skipping already applied migration: 20251022.sql
Skipping already applied migration: 20251023.sql
Skipping already applied migration: 20251029.sql
Skipping already applied migration: 20251101.sql
Skipping already applied migration: 20251103.sql
Skipping already applied migration: 20251104.sql
Skipping already applied migration: 20251107.sql
Skipping already applied migration: 20251110.sql
Skipping already applied migration: 20251111.sql
Skipping already applied migration: 20251112.sql
Existing wxo_observability DB found
Skipping already applied observability migration: 00_init_db.sql
Skipping already applied observability migration: 01_create_tables.sql
Skipping already applied observability migration: 02_triggers_and_functions.sql
Skipping already applied observability migration: 03_security.sql
Skipping already applied observability migration: 04_rls_policies.sql
Skipping already applied observability migration: 20250116.sql
[INFO] - Migration ran successfully.
[INFO] - Waiting for orchestrate server to be fully initialized and ready...
[INFO] - Orchestrate services initialized successfully
[INFO] - local tenant found
[INFO] - You can run 'orchestrate env activate local' to set your environment or 'orchestrate chat start' to start the UI service and begin chatting.
[INFO] - You can access the observability platform Langfuse at http://localhost:3010, username: orchestrate@ibm.com, password: orchestrate
ADK >orchestrate env activate local
[INFO] - local tenant found
[INFO] - Environment 'local' is now active
ADK >orchestrate connections add --app-id Tavily
```

2. After receiving the success confirmation message, type the following command to list the existing connections on your local environment (A)

```
orchestrate connections list
```

```
Skipping already applied migration: 20251103.sql
Skipping already applied migration: 20251104.sql
Skipping already applied migration: 20251107.sql
Skipping already applied migration: 20251110.sql
Skipping already applied migration: 20251111.sql
Skipping already applied migration: 20251112.sql
Existing wxo_observability DB found
Skipping already applied observability migration: 00_init_db.sql
Skipping already applied observability migration: 01_create_tables.sql
Skipping already applied observability migration: 02_triggers_and_functions.sql
Skipping already applied observability migration: 03_security.sql
Skipping already applied observability migration: 04_rls_policies.sql
Skipping already applied observability migration: 20250116.sql
[INFO] - Migration ran successfully.
[INFO] - Waiting for orchestrate server to be fully initialized and ready...
[INFO] - Orchestrate services initialized successfully
[INFO] - local tenant found
[INFO] - You can run 'orchestrate env activate local' to set your environment or 'orchestrate chat start' to start the UI service and begin chatting.
[INFO] - You can access the observability platform Langfuse at http://localhost:3010, username: orchestrate@ibm.com, password: orchestrate
ADK >orchestrate env activate local
[INFO] - local tenant found
[INFO] - Environment 'local' is now active
ADK >orchestrate connections add --app-id Tavily
[INFO] - Creating connection 'Tavily'
[INFO] - Successfully created connection 'Tavily'
ADK >orchestrate connections list
```

3. Note your Tavily connection has been created (A). As you are running on watsonx Orchestrator Developer Edition, only Draft environments are supported. This is why no live connection is listed.

```
Skipping already applied observability migration: 01_create_tables.sql
Skipping already applied observability migration: 02_triggers_and_functions.sql
Skipping already applied observability migration: 03_security.sql
Skipping already applied observability migration: 04_rls_policies.sql
Skipping already applied observability migration: 20250116.sql
[INFO] - Migration ran successfully.
[INFO] - Waiting for orchestrate server to be fully initialized and ready...
[INFO] - Orchestrator services initialized successfully
[INFO] - local tenant found
[INFO] - You can run 'orchestrate env activate local' to set your environment or 'orchestrate chat start' to start the UI service and begin chatting.
[INFO] - You can access the observability platform Langfuse at http://localhost:3010, username: orchestrate@ibm.com, password: orchestrate
ADK >orchestrate env activate local
[INFO] - local tenant found
[INFO] - Environment 'local' is now active
ADK >orchestrate connections add --app-id Tavily
[INFO] - Creating connection 'Tavily'
[INFO] - Successfully created connection 'Tavily'
ADK >orchestrate connections list
Draft
```

App ID	Auth Type	Type	Credentials Set/ Connected
Tavily	n/a	n/a	✗



Note: You can switch to a SaaS environment to create the MCP tools if you want to work with Draft and Live environments. To learn how to switch to a SaaS environment, please refer to the ADK – Using ADK with SaaS lab. You can then repeat the following instructions replacing Draft with live.

Configure the environments parameters

Next you will configure the connection parameters as you did in the previous part through the UI:

- You will configure the **draft** env
- It will use **key_value** pairs
- Credentials will be used for each **team** members

1. Enter the following command to configure the connection (A).

```
orchestrate connections configure --app-id Tavily --env draft --kind key_value
--type team
```

```
Skipping already applied migration: 20251110.sql
Skipping already applied migration: 20251111.sql
Skipping already applied migration: 20251112.sql
Existing wxo_observability DB found
Skipping already applied observability migration: 00_init_db.sql
Skipping already applied observability migration: 01_create_tables.sql
Skipping already applied observability migration: 02_triggers_and_functions.sql
Skipping already applied observability migration: 03_security.sql
Skipping already applied observability migration: 04_rls_policies.sql
Skipping already applied observability migration: 20250116.sql
[INFO] - Migration ran successfully.
[INFO] - Waiting for orchestrate server to be fully initialized and ready...
[INFO] - Orchestrator services initialized successfully
[INFO] - local tenant found
[INFO] - You can run 'orchestrate env activate local' to set your environment or 'orchestrate chat start' to start the UI service and begin chatting.
[INFO] - You can access the observability platform Langfuse at http://localhost:3010, username: orchestrate@ibm.com, password: orchestrate
ADK >orchestrate env activate local
[INFO] - local tenant found
[INFO] - Environment 'local' is now active
ADK >orchestrate connections add --app-id Tavily
[INFO] - Creating connection 'Tavily'
[INFO] - Successfully created connection 'Tavily'
ADK >orchestrate connections list
Draft
```

App ID	Auth Type	Type	Credentials Set/ Connected
Tavily	n/a	n/a	✗

```
ADK >orchestrate connections configure --app-id Tavily --env draft --kind key_value --type team
```



2. After receiving the success notification message type 'orchestrate connections list' again (A).

```
wxo-ADK --zsh -- 160x38

Skipping already applied migration: 20251029.sql
Skipping already applied migration: 20251101.sql
Skipping already applied migration: 20251103.sql
Skipping already applied migration: 20251104.sql
Skipping already applied migration: 20251107.sql
Skipping already applied migration: 20251110.sql
Skipping already applied migration: 20251111.sql
Skipping already applied migration: 20251112.sql
Existing wxo_observability DB found
Skipping already applied observability migration: 00_init_db.sql
Skipping already applied observability migration: 01_create_tables.sql
Skipping already applied observability migration: 02_triggers_and_functions.sql
Skipping already applied observability migration: 03_security.sql
Skipping already applied observability migration: 04_rls_policies.sql
Skipping already applied observability migration: 20250116.sql
[INFO] - Migration ran successfully.
[INFO] - Waiting for orchestrate server to be fully initialized and ready...
[INFO] - Orchestrate services initialized successfully
[INFO] - local tenant found
[INFO] - You can run 'orchestrate env activate local' to set your environment or 'orchestrate chat start' to start the UI service and begin chatting.
[INFO] - You can access the observability platform Langfuse at http://localhost:3010, username: orchestrate@ibm.com, password: orchestrate
ADK >orchestrate env activate local
[INFO] - local tenant found
[INFO] - Environment 'local' is now active
ADK >orchestrate connections add --app-id Tavily
[INFO] - Creating connection 'Tavily'
[INFO] - Successfully created connection 'Tavily'
ADK >orchestrate connections list
Draft


| App ID | Auth Type | Type | Credentials Set/ Connected |
|--------|-----------|------|----------------------------|
| Tavily | n/a       | n/a  | ✗                          |


ADK >orchestrate connections configure --app-id Tavily --env draft --kind key_value --type team
[INFO] - Creating configuration for connection 'Tavily' in the 'draft' environment
[INFO] - Configuration successfully created for 'draft' environment of connection 'Tavily'.
ADK >orchestrate connections list
```

3. Note that the connection parameters are now set (A). Next you will provide the credentials to connect your Tavily instance.

```
wxo-ADK --zsh -- 160x38

Skipping already applied migration: 20251112.sql
Existing wxo_observability DB found
Skipping already applied observability migration: 00_init_db.sql
Skipping already applied observability migration: 01_create_tables.sql
Skipping already applied observability migration: 02_triggers_and_functions.sql
Skipping already applied observability migration: 03_security.sql
Skipping already applied observability migration: 04_rls_policies.sql
Skipping already applied observability migration: 20250116.sql
[INFO] - Migration ran successfully.
[INFO] - Waiting for orchestrate server to be fully initialized and ready...
[INFO] - Orchestrate services initialized successfully
[INFO] - local tenant found
[INFO] - You can run 'orchestrate env activate local' to set your environment or 'orchestrate chat start' to start the UI service and begin chatting.
[INFO] - You can access the observability platform Langfuse at http://localhost:3010, username: orchestrate@ibm.com, password: orchestrate
ADK >orchestrate env activate local
[INFO] - local tenant found
[INFO] - Environment 'local' is now active
ADK >orchestrate connections add --app-id Tavily
[INFO] - Creating connection 'Tavily'
[INFO] - Successfully created connection 'Tavily'
ADK >orchestrate connections list
Draft


| App ID | Auth Type | Type | Credentials Set/ Connected |
|--------|-----------|------|----------------------------|
| Tavily | n/a       | n/a  | ✗                          |


ADK >orchestrate connections configure --app-id Tavily --env draft --kind key_value --type team
[INFO] - Creating configuration for connection 'Tavily' in the 'draft' environment
[INFO] - Configuration successfully created for 'draft' environment of connection 'Tavily'.
ADK >orchestrate connections list
Draft


| App ID | Auth Type       | Type | Credentials Set/ Connected |
|--------|-----------------|------|----------------------------|
| Tavily | key_value_creds | team | ✗                          |

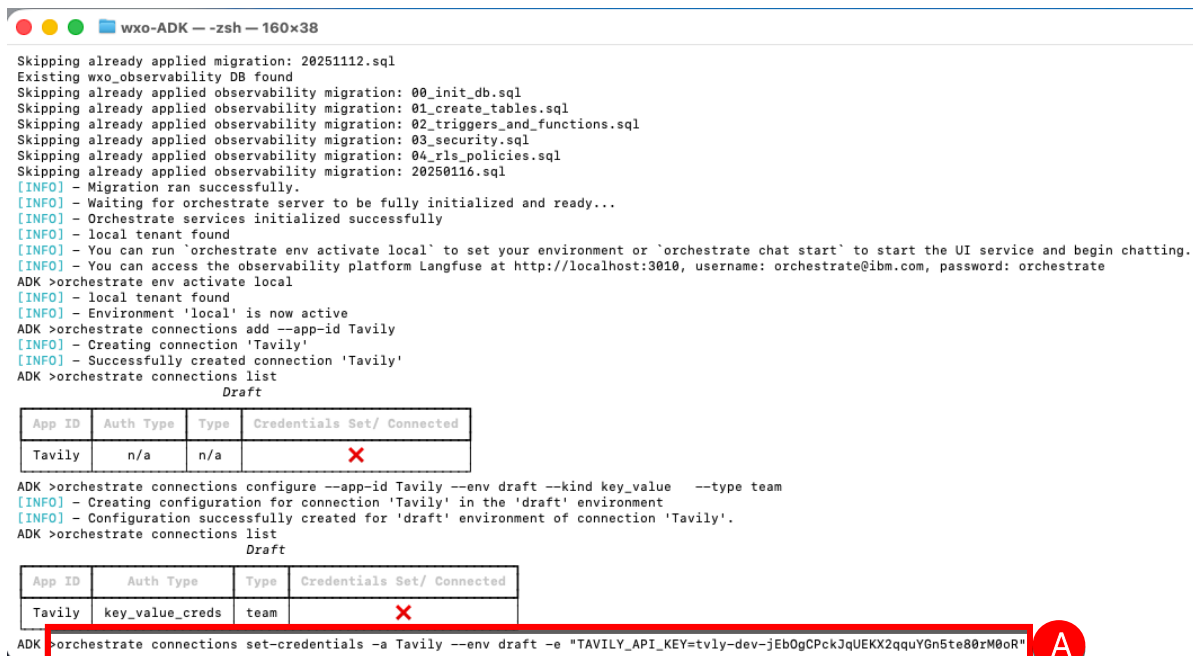

ADK >
```

Specify the credentials to connect your tavily server

The authentication method is to use key_value pairs. You have already configured your Tavily sever in the Preparation guide. You must now provide this key to the Tavily connection you have created.

4. Enter the following command replacing <your Tavily api key> with your own api key collected in item 11 of the prep guide (A)

```
orchestrate connections set-credentials -a Tavily --env draft -e
"TAVILY_API_KEY=<your tavily api key>"
```



```
wxo-ADK --zsh -- 160x38
Skipping already applied migration: 20251112.sql
Existing wxo_observability DB found
Skipping already applied observability migration: 00_init_db.sql
Skipping already applied observability migration: 01_create_tables.sql
Skipping already applied observability migration: 02_triggers_and_functions.sql
Skipping already applied observability migration: 03_security.sql
Skipping already applied observability migration: 04_rls_policies.sql
Skipping already applied observability migration: 20250116.sql
[INFO] - Migration ran successfully.
[INFO] - Waiting for orchestrate server to be fully initialized and ready...
[INFO] - Orchestrate services initialized successfully
[INFO] - local tenant found
[INFO] - You can run 'orchestrate env activate local' to set your environment or 'orchestrate chat start' to start the UI service and begin chatting.
[INFO] - You can access the observability platform Langfuse at http://localhost:3010, username: orchestrate@ibm.com, password: orchestrate
ADK >orchestrate env activate local
[INFO] - local tenant found
[INFO] - Environment 'local' is now active
ADK >orchestrate connections add --app-id Tavily
[INFO] - Creating connection 'Tavily'
[INFO] - Successfully created connection 'Tavily'
ADK >orchestrate connections list
Draft

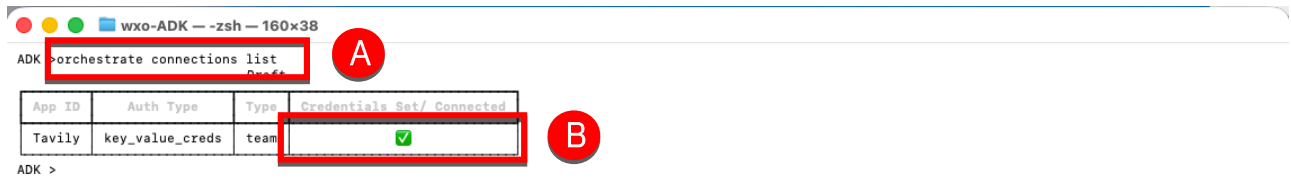

| App ID | Auth Type | Type | Credentials Set/ Connected |
|--------|-----------|------|----------------------------|
| Tavily | n/a       | n/a  | ✗                          |


ADK >orchestrate connections configure --app-id Tavily --env draft --kind key_value --type team
[INFO] - Creating configuration for connection 'Tavily' in the 'draft' environment
[INFO] - Configuration successfully created for 'draft' environment of connection 'Tavily'.
ADK >orchestrate connections list
Draft


| App ID | Auth Type       | Type | Credentials Set/ Connected |
|--------|-----------------|------|----------------------------|
| Tavily | key_value_creds | team | ✗                          |


ADK >orchestrate connections set-credentials -a Tavily --env draft -e "TAVILY_API_KEY=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"
```

5. After receiving the success notification message, type again 'orchestrate connections list' (A) and note that the connection is now completely configured and connected (B).

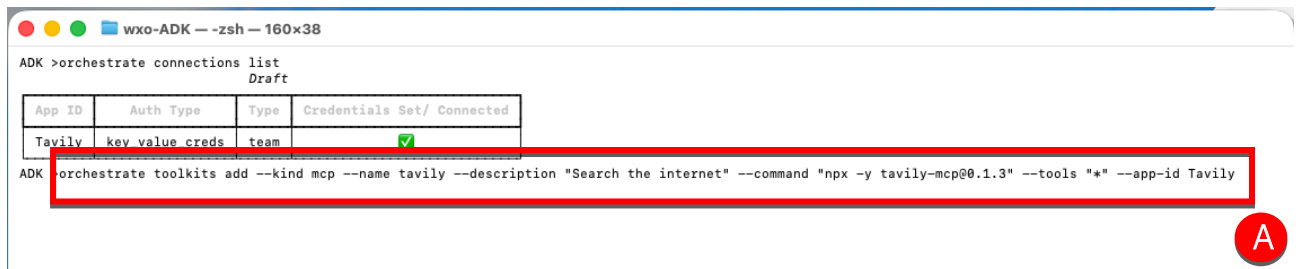


Import Tavily tools in a Tavily toolkit

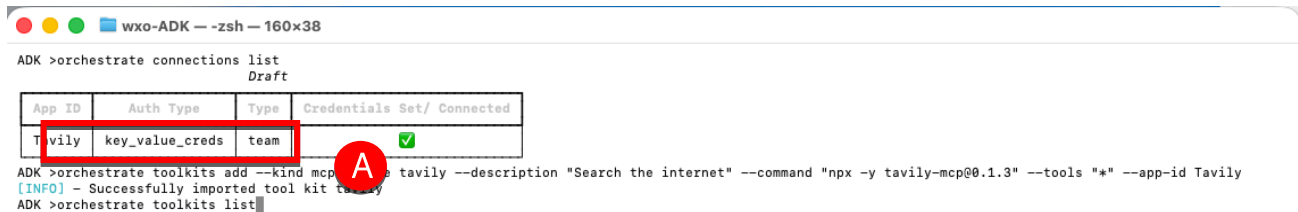
Next you will import **mcp kind** search tool. The tool to import from Tavily is “**Tavily-search**” and the **npx -y tavily-mcp@0.1.3** command will be used to import the tool locally. The tool description will be “**Search the internet**” in this lab.

1. Type the following command to import all the Tavily tools in your tavily toolkit.

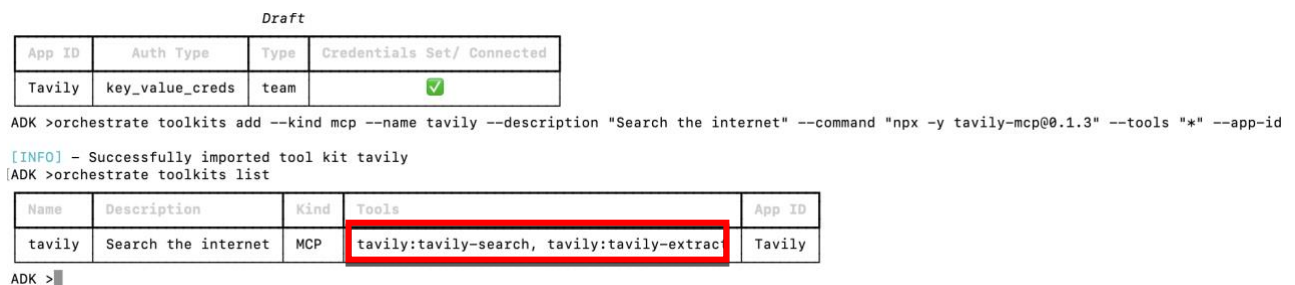
```
orchestrate toolkits add --kind mcp --name tavily --description "Search the internet" --command "npx -y tavily-mcp@0.1.3" --tools "*" --app-id Tavily
```



2. Wait for the toolkit creation to be performed then type 'orchestrate toolkits list' (A).



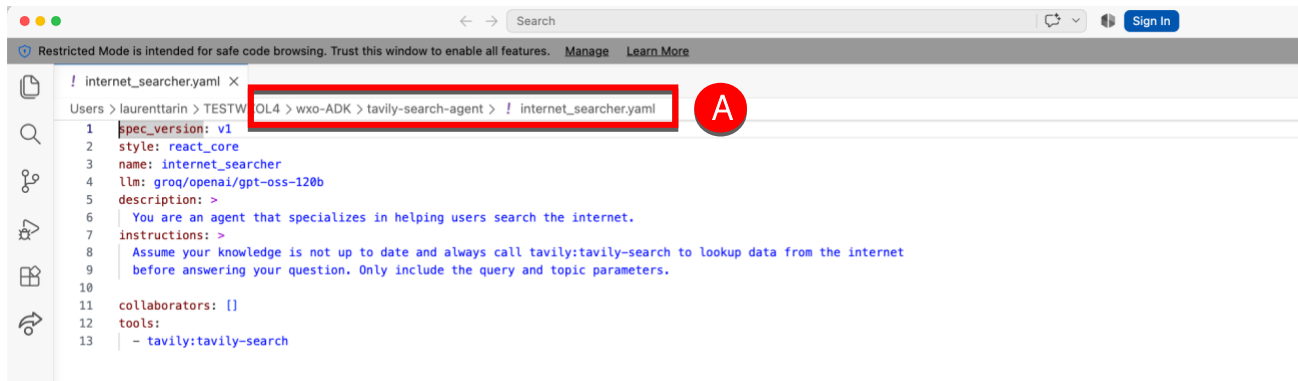
The toolkit is ready, and it contains the tavily:tavily-search tool as expected. You can now create the agent that will use this search tool.



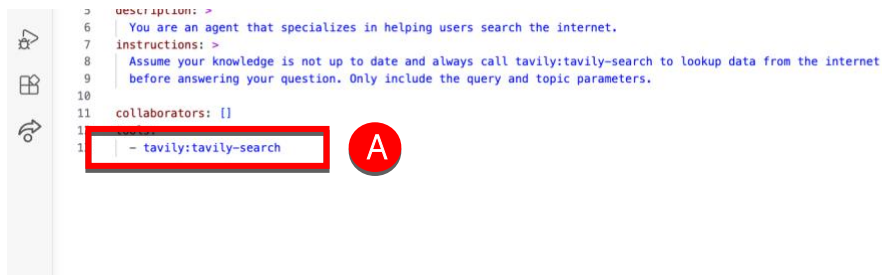
Create the Tavily agent

The Tavily_search agent is described in the internet_searcher.yaml file contained in your workspace.

1. Edit the `wxo-ADK/tavily-search-agent/internet_searcher.yaml` file (A) using your preferred text editor.

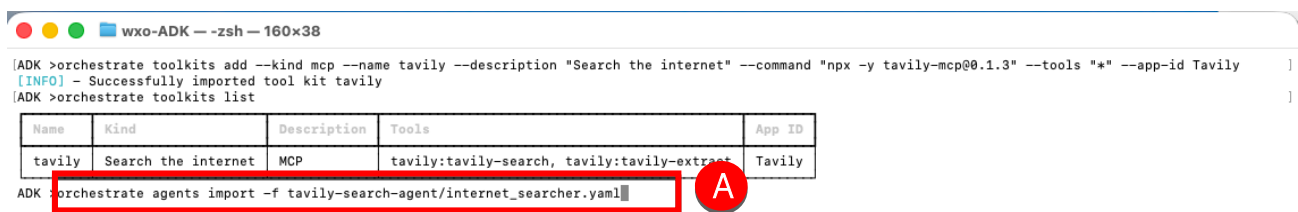


2. Review the agent and note that it will use the Tavily-search tool you have imported from Tavily (A).



3. Type the following command in the terminal to import the agent (A).

```
orchestrate agents import -f tavily-search-agent/internet_searcher.yaml
```



Preview the agent

1. Type the following command to start the chat interface (A).

```
Orchestrate chat start
```

```
wxo-ADK -- zsh -- 160x38

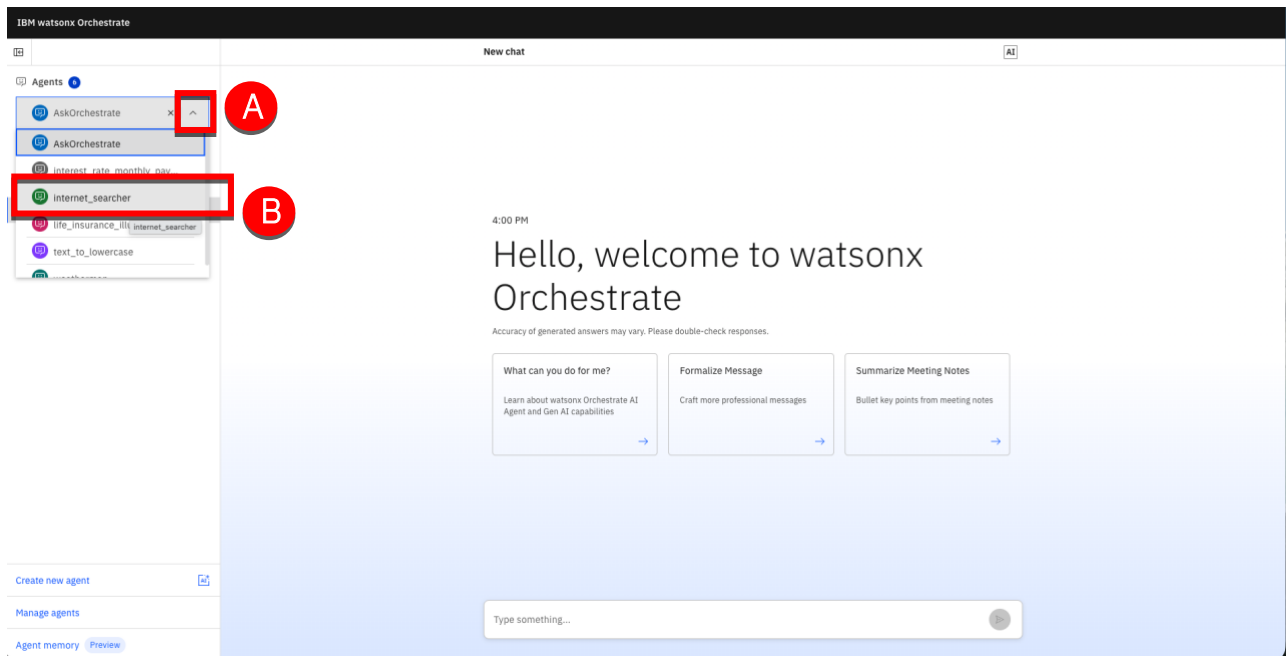
[ADK >orchestrate toolkits add --kind mcp --name tavily --description "Search the internet" --command "npx -y tavily-mcp@0.1.3" --tools "*" --app-id Tavily]
[INFO] - Successfully imported tool kit tavily
[ADK >orchestrate toolkits list]

Name Kind Description Tools App ID
tavily Search the internet MCP tavily:tavily-search, tavily:tavily-extract Tavily

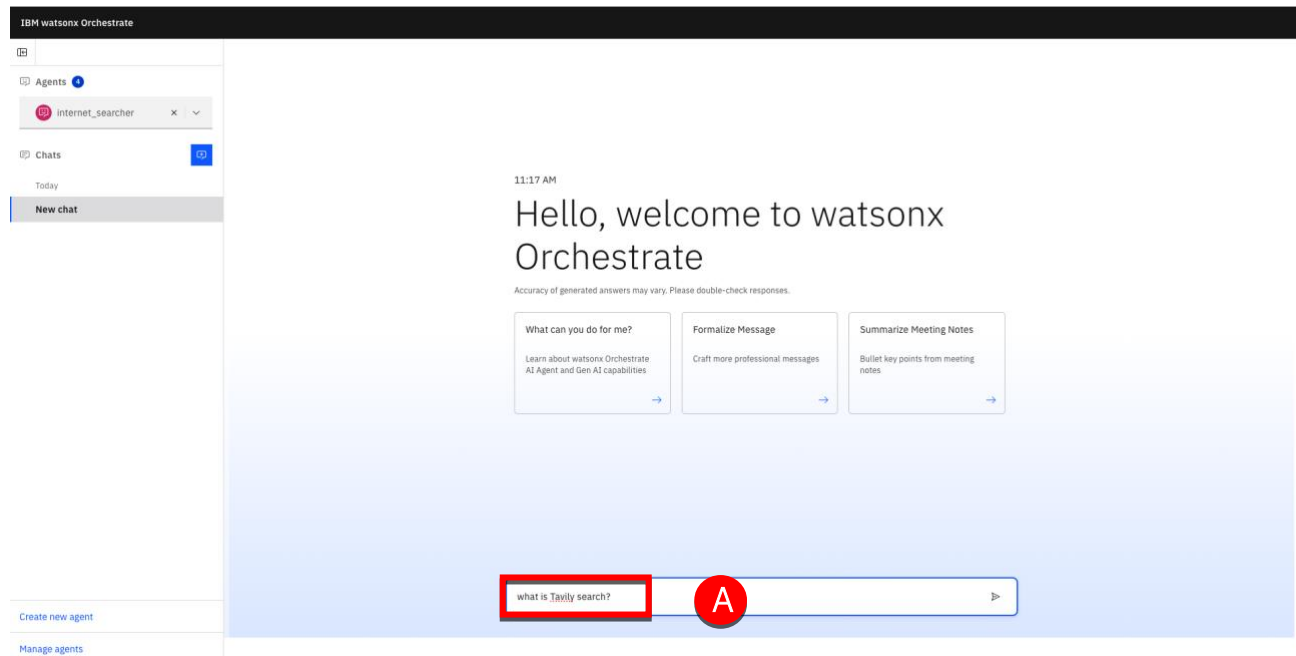
[ADK >orchestrate agents import -f tavily-search-agent/internet_searcher.yaml]
[INFO] - Agent internet_searcher imported successfully
[ADK >Orchestrate chat start]
```

A

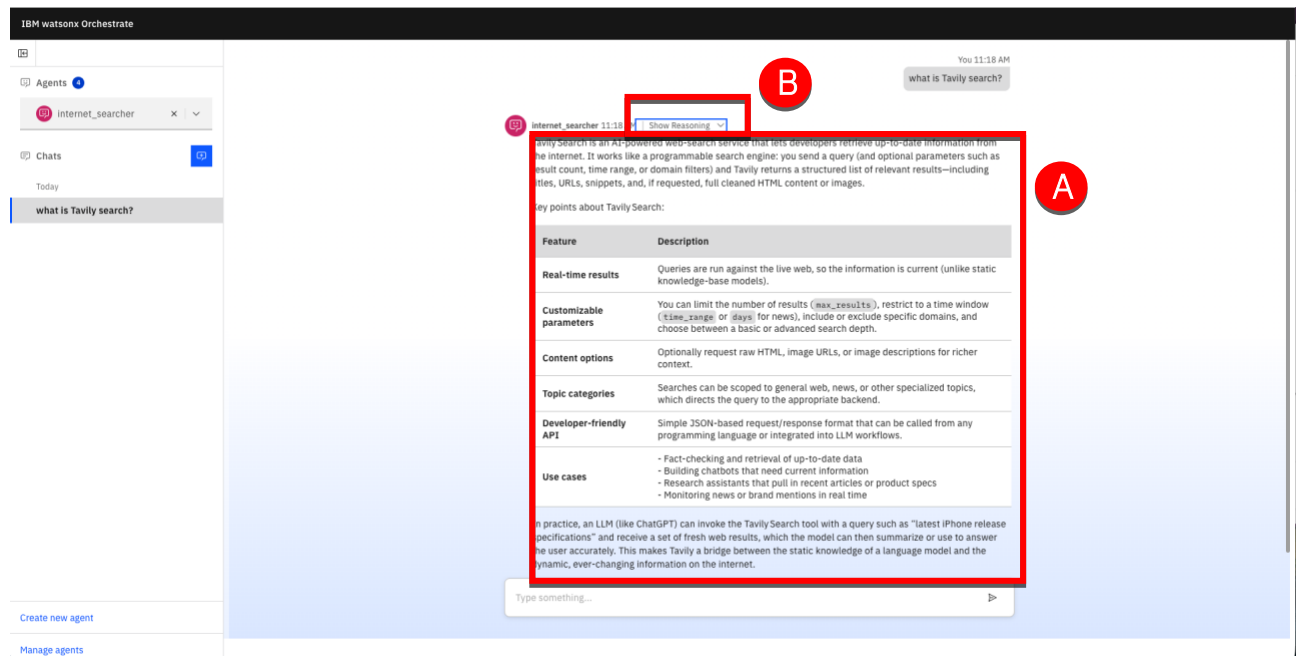
2. Expand the agent selector (A) and select the **internet_searcher** agent (B) from the list.



3. Type 'what is Tavily search?' in the chat barre (A).



4. Note the response (A) then expand the **Show reasoning** menu (B).



5. Expand Step 1 (A)

IBM watsonx Orchestrate

Agents: internet_searcher

Chats: what is Tavily search?

Today

what is Tavily search?

You 11:18 AM

internet_searcher 11:18 AM | Hide Reasoning

Step 1

Tavily Search is an AI-powered web search engine that lets developers retrieve up-to-date information from the internet. It works like a programmatic search engine: you send a query (and optional parameters such as result count, time range, or domain filters) and Tavily returns a structured list of relevant results—including titles, URLs, snippets, and, if requested, full cleaned HTML content or images.

Key points about Tavily Search:

Feature	Description
Real-time results	Queries are run against the live web, so the information is current (unlike static knowledge-base models).
Customizable parameters	You can limit the number of results (<code>max_results</code>), restrict to a time window (<code>time_range</code> or <code>days</code> for news), include or exclude specific domains, and choose between a basic or advanced search depth.
Content options	Optionally request raw HTML, image URLs, or image descriptions for richer context.
Topic categories	Searches can be scoped to general web, news, or other specialized topics, which directs the query to the appropriate backend.
Developer-friendly API	Simple JSON-based request/response format that can be called from any programming language or integrated into LLM workflows.
Use cases	- Fact-checking and retrieval of up-to-date data - Building chatbots that need current information - Research assistants that pull in recent articles or product specs - Monitoring news or brand mentions in real time

In practice, an LLM (like ChatGPT) can invoke the Tavily Search tool with a query such as "latest iPhone release"

Type something...

6. You can now review in detail how the Tavily search engine was invoked (A), how the response was sent back by the server and how the final answer was generated by the LLM (B).

IBM watsonx Orchestrate

Agents: internet_searcher

Chats: What is Tavily?

Today

What is Tavily?

You 11:45 AM

internet_searcher 11:45 AM | Hide Reasoning

Step 1

Tool: tavily-search

Input - 5 Lines

```
{
  "max_results": 5,
  "query": "what is Tavily",
  "topic": "general"
}
```

Output - 3 Lines

```
{
  "data": {
    "meta": {
      "content": [
        {
          "type": "text",
          "text": "Detailed Results:\n\nTitle: Tavily Search - Docs by LangChain\nURL: https://docs.langchain.com/oss/javascript/integrations/tools/tavily_search\nContent: # Tavily Search. Tavily is a search engine built specifically for AI agents (LLMs), delivering real-time, accurate, and factual results at speed. Tavily offers two key endpoints, one of which being Search, which provides search results tailored for LLMs and RAG. This guide provides a quick overview for getting started with the Tavily tool. For a complete breakdown of the Tavily tool, you can find more detailed documentation in the API reference. ## Overview. ### Integration details. The integration lives in the @langchain/tavily package, which you can install as shown below: npm install @langchain/tavily @langchain/core. ## Installation\nimport { TavilySearch } from"
        }
      ]
    }
  }
}
```

Type something...

Next step

Congratulations, you have reached the end of this lab. In a later lab you will learn how to use the ADK and SaaS environment together and connect your ADK CLI to the SaaS environment so that CLI commands are run against the SaaS environment.

As a next step you can:

- Play with the parameters of the Taviliy search tool to see the impact of the query result and study how the agent will manage these parameters
- Import new tools from other MCP servers that you can find here:
 - <https://github.com/appcypher/awesome-mcp-servers>
 - <https://github.com/modelcontextprotocol/servers>

Summary

Add a section at the end to summarize what the user accomplished from the lab.

In this lab you have:

- Created an admin account
- Installed the product
- Tested the access
- Verified the setup

Appendix: Troubleshooting

Unable to create a connection

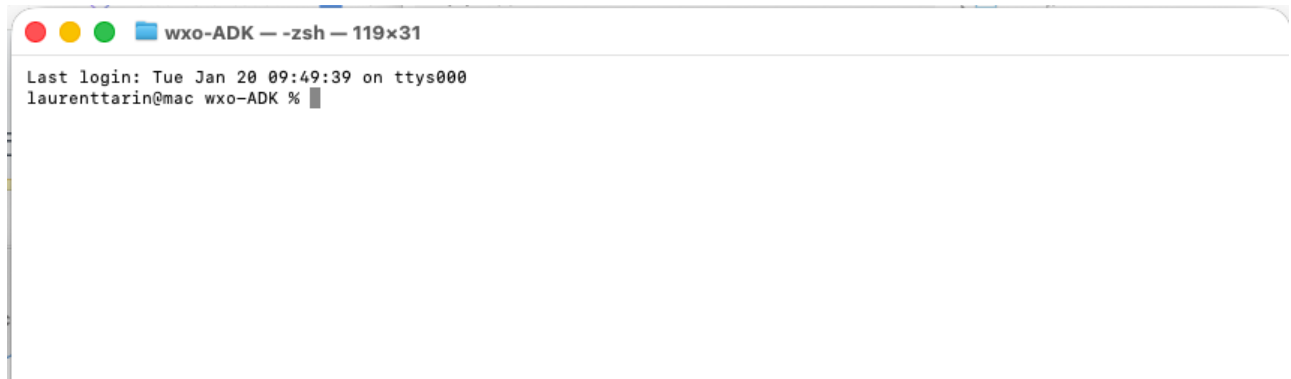
If you are unable to create a connection using watsonx Orchestrate running on IBM Cloud, try using a private browser window.

If the problem persists your SaaS environment may still be provisioning, if after an hour you are still unable to create a connection, delete the reservation and reserve another environment.

Appendix: Reset your environment

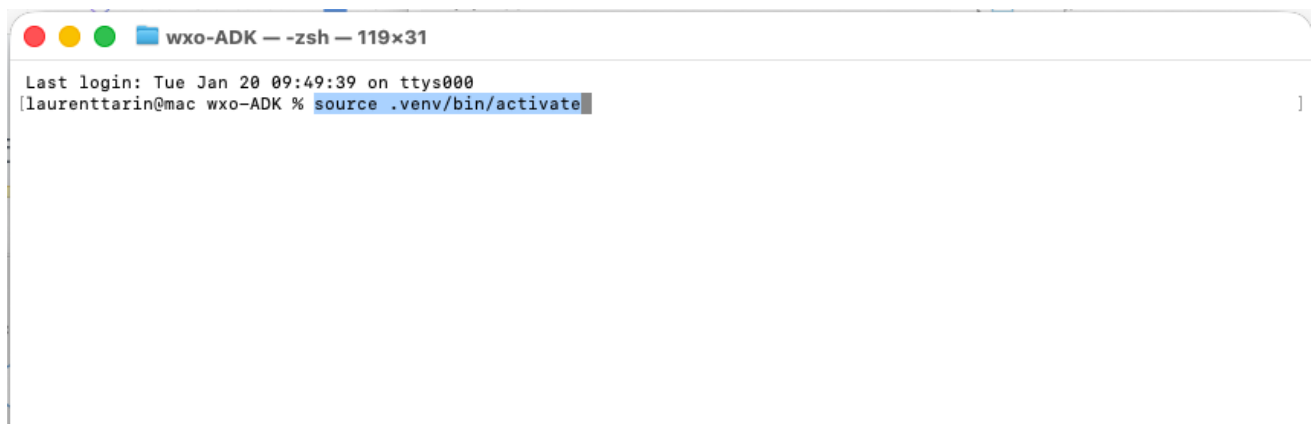
These steps will remove any existing tools and agents from your environment and restart the local watsonx Orchestrate server in readiness to start the lab.

1. Open a Terminal window on your **wxO-ADK** folder (A).



2. Enter the following command into the terminal to start your virtual environment:

```
source .venv/bin/activate
```



3. Check that the prompt starts with (.venv) (A) indicating that you are in your virtual environment.

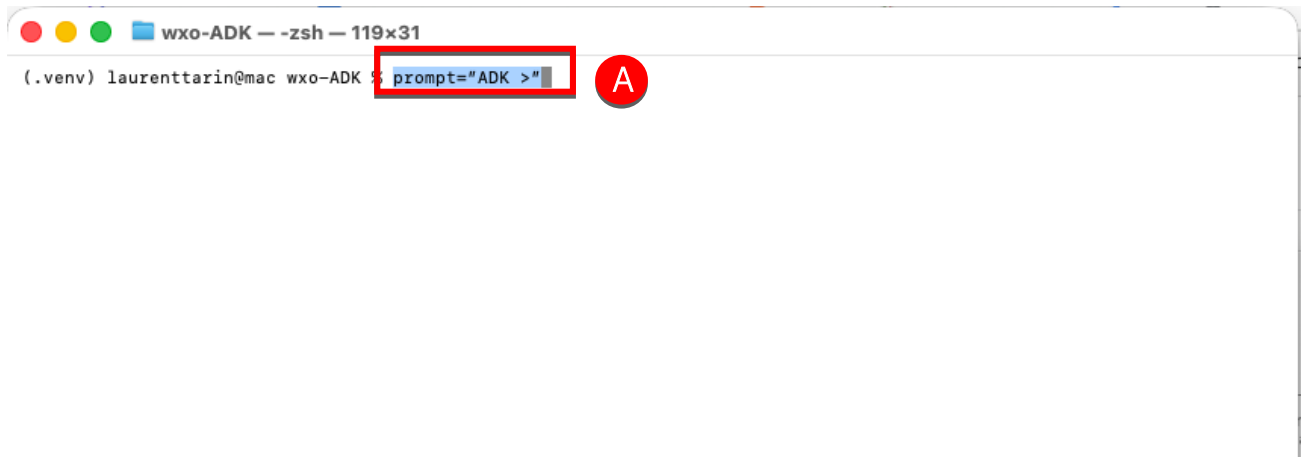


A terminal window titled 'wxo-ADK - zsh - 119x31'. The output shows the last login time and the execution of the 'source .venv/bin/activate' command, which changes the prompt to '(.venv) laurenttarin@mac wxo-ADK %'.

```
Last login: Tue Jan 20 09:49:39 on ttys000
[laurenttarin@mac wxo-ADK % source .venv/bin/activate
(.venv) laurenttarin@mac wxo-ADK %
```

4. Enter the following command into the terminal to improve the command visibility, then press Enter:

```
MS Windows users: function prompt{"ADK>"}
Mac users: prompt="ADK >"
```



A terminal window titled 'wxo-ADK - zsh - 119x31'. The prompt is '(.venv) laurenttarin@mac wxo-ADK \$'. The command 'prompt="ADK >"' is entered and highlighted with a red box. A red circle with the letter 'A' is next to the command.

```
(.venv) laurenttarin@mac wxo-ADK $ prompt="ADK >"
```

5. Enter the following command into the terminal to stop your chat service, then press Enter.

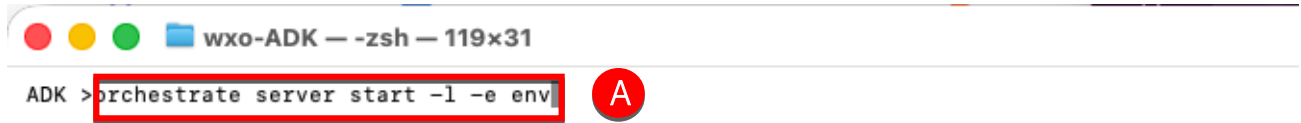
```
orchestrate chat stop
```

6. Enter the following command into the terminal to reset your server, then press Enter.

```
orchestrate server reset
```

7. Enter the following command into the terminal to start the server with your env configuration, then press Enter.

```
orchestrate server start -l -e env
```



Note: Some services may fail to start on the first attempt, and you may encounter issues connecting to the docker repository. Retry the start command in this situation, depending on the volume of requests, it can take several attempts to connect.

When startup is complete the terminal output will resemble the example below, restart can take 2-3 minutes to complete. The local server will be ready when you will see this **INFO** in the terminal screen (A):

```
wxo-ADK -- -zsh -- 119x31

Skipping already applied migration: 20251001.sql
Skipping already applied migration: 20251006.sql
Skipping already applied migration: 20251007.sql
Skipping already applied migration: 20251008.sql
Skipping already applied migration: 20251009.sql
Skipping already applied migration: 20251022.sql
Skipping already applied migration: 20251023.sql
Skipping already applied migration: 20251029.sql
Skipping already applied migration: 20251101.sql
Skipping already applied migration: 20251103.sql
Skipping already applied migration: 20251104.sql
Skipping already applied migration: 20251107.sql
Skipping already applied migration: 20251110.sql
Skipping already applied migration: 20251111.sql
Skipping already applied migration: 20251112.sql
Existing wxo_observability DB found
Skipping already applied observability migration: 00_init_db.sql
Skipping already applied observability migration: 01_create_tables.sql
Skipping already applied observability migration: 02_triggers_and_functions.sql
Skipping already applied observability migration: 03_security.sql
Skipping already applied observability migration: 04_rls_policies.sql
Skipping already applied observability migration: 20250116.sql
[INFO] - Migration ran successfully.
[INFO] - Waiting for orchestrate server to be fully initialized and ready...
[INFO] - Orchestrate services initialized successfully
[INFO] - local tenant found
[INFO] - You can run 'orchestrate env activate local' to set your environment or 'orchestrate chat start' to start the
UI service and begin chatting.
[INFO] - You can access the observability platform Langfuse at http://localhost:3010, username: orchestrate@ibm.com, pa
ssword: orchestrate
ADK
```



You are now ready to work with watsonx Orchestrate ADK.

